

R2P Program Unit 3 - Lesson 3

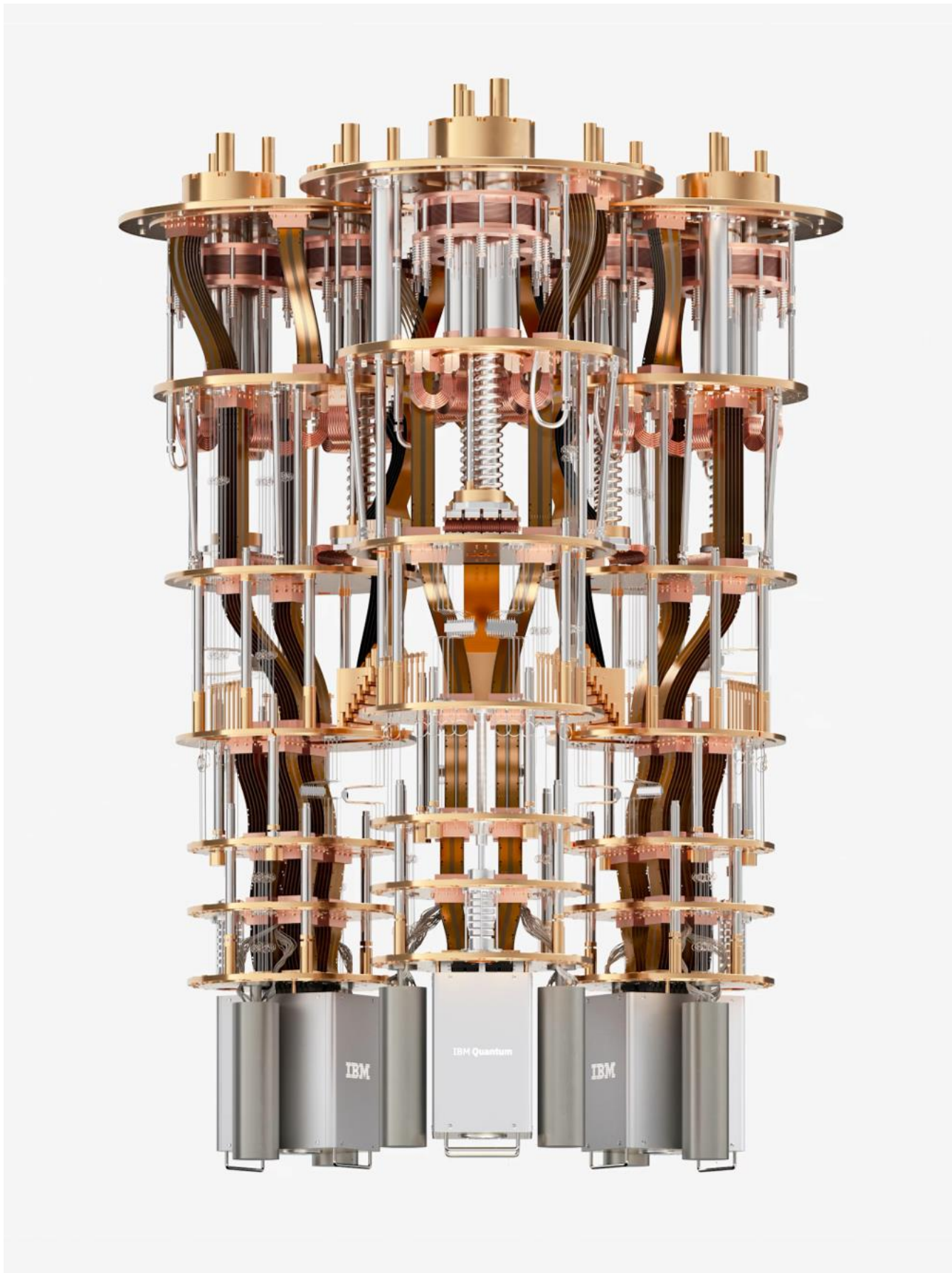
Advanced Techniques

Sean Wagner, Ph.D.

Research Scientist | Quantum Technical Ambassador

IBM

Table of contents



Part 1

Advanced Error Mitigation

Noise Learning
Samplomatic and the executor primitive

Part 2

Mid Circuit Measurements

Utility Scale Dynamic Circuits

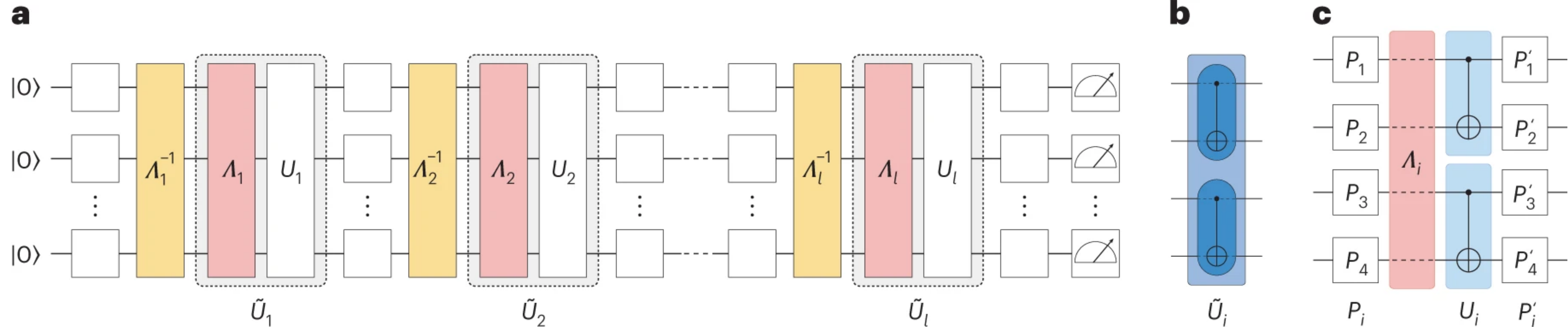
Part 3

Hands-on

Long Range Entanglement using Dynamic Circuits

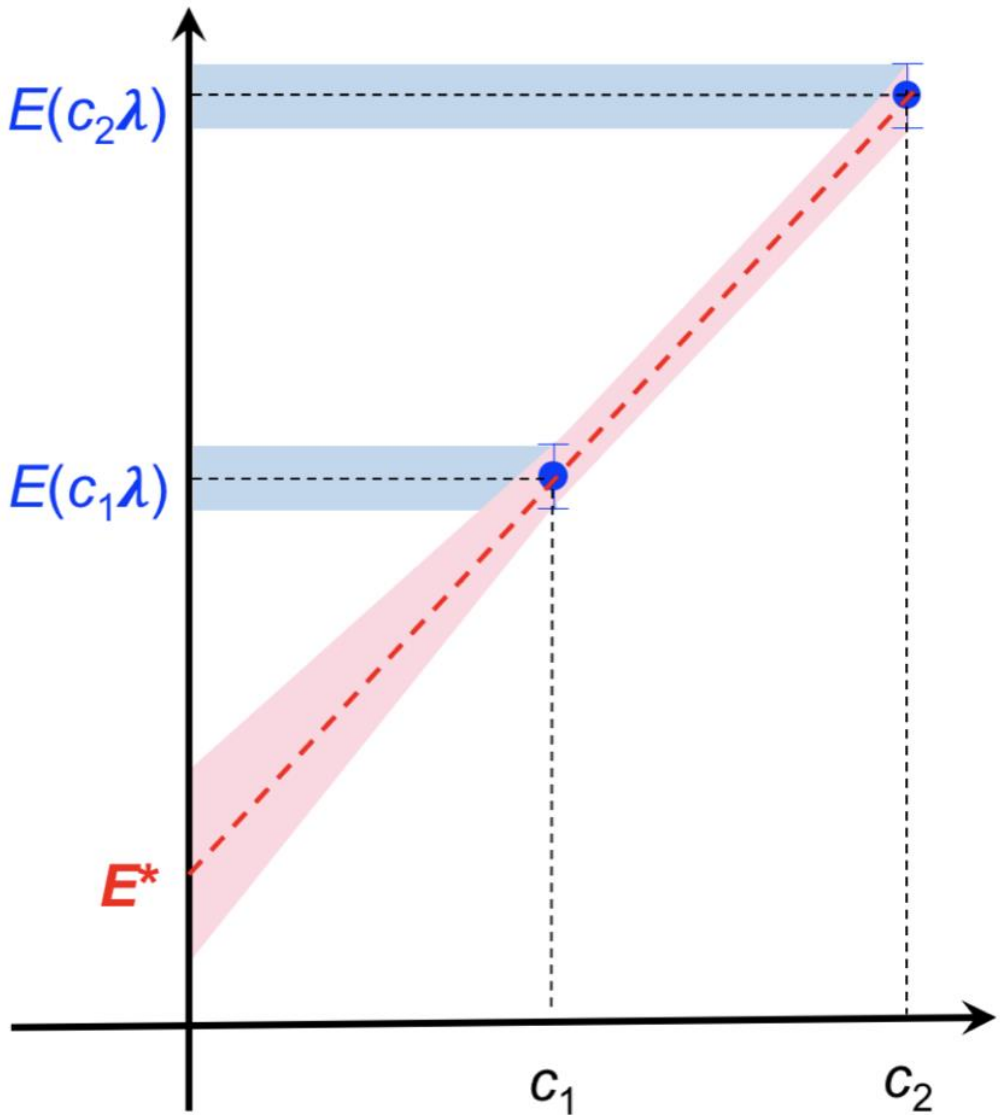
Noise Learning

Learning Noise



- We learn noise in layers, focusing on 2 qubit gates.
- We can abstract the noise from the noisy gates with the help of Pauli Twirling.
- In the ideal case we could just use the inverse of the learned noise to cancel it.

Use of Noise Learning: Zero Noise Extrapolation



Probabilistic Error Amplification

Add noise via sampling Pauli channels

error simulation

time

control

target

Twirling

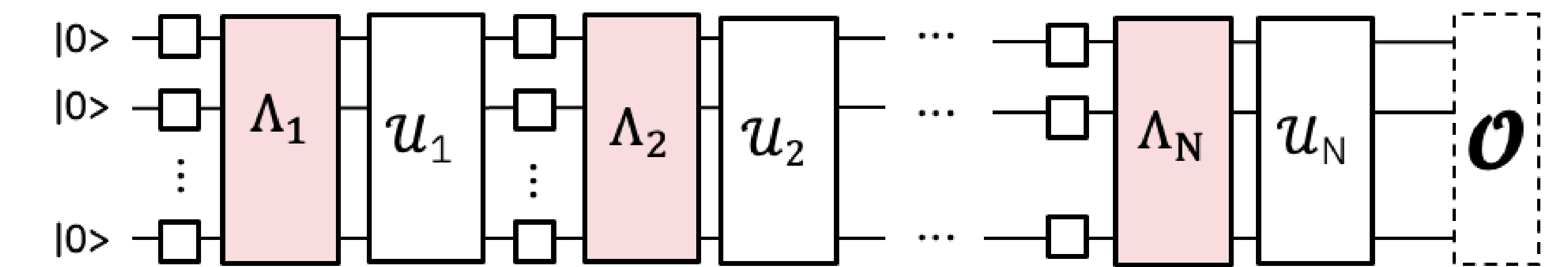
Raw operation

Twirling

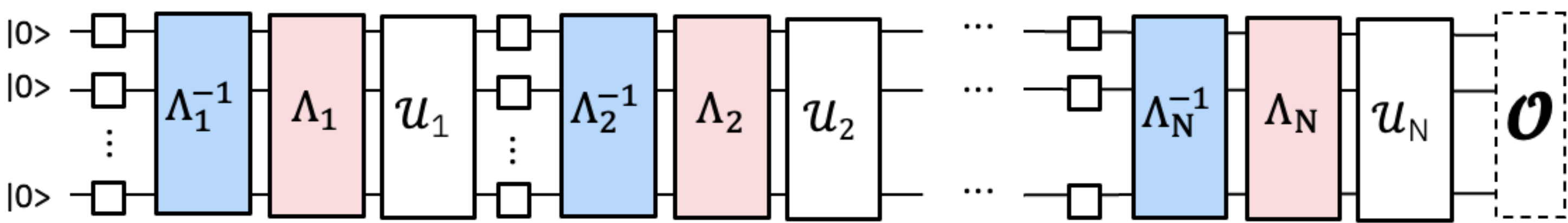
Simulation

Li & Benjamin PRX (2017)

Probabilistic Error Cancellation



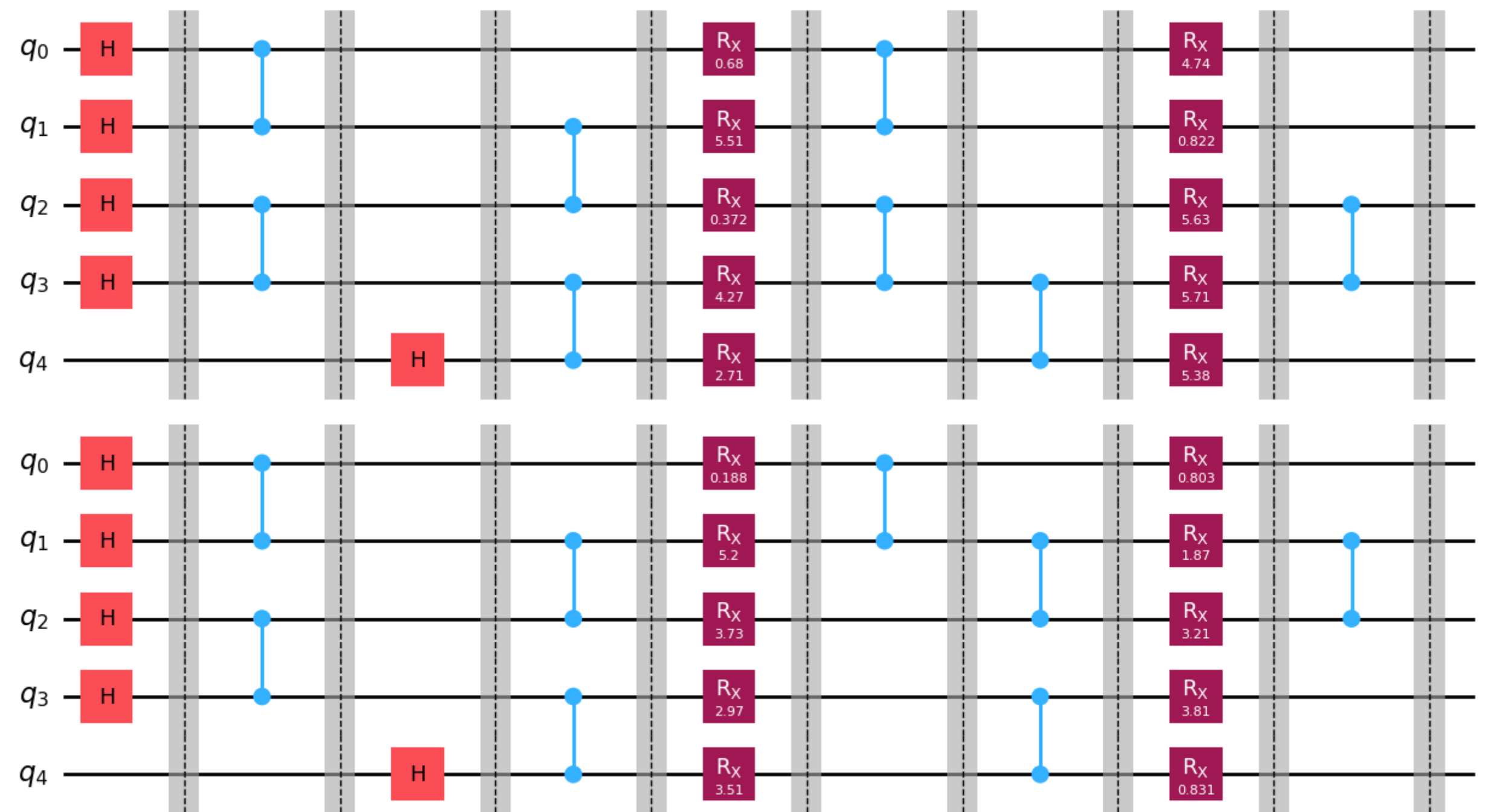
$$\mathcal{O}_{\text{ideal}} = \sum_i \eta_i \mathcal{O}_{\text{noisy},i}$$



Noise Learner and Noise Layers

- 2 Qubit layer noise is learned single qubit gates are ignored.
- Repeated layers are only learned once, this can be used for optimization.
- Beware: It takes a lot of calculation time to learn the noise of a circuit!
- You can use barriers to force layers to ensure you have repeated ones.
- Having to rely on barriers feels not efficient -> We need a better tool.

```
# run the noise learner job
learner = NoiseLearner(backend, options)
job = learner.run(circuits)
```

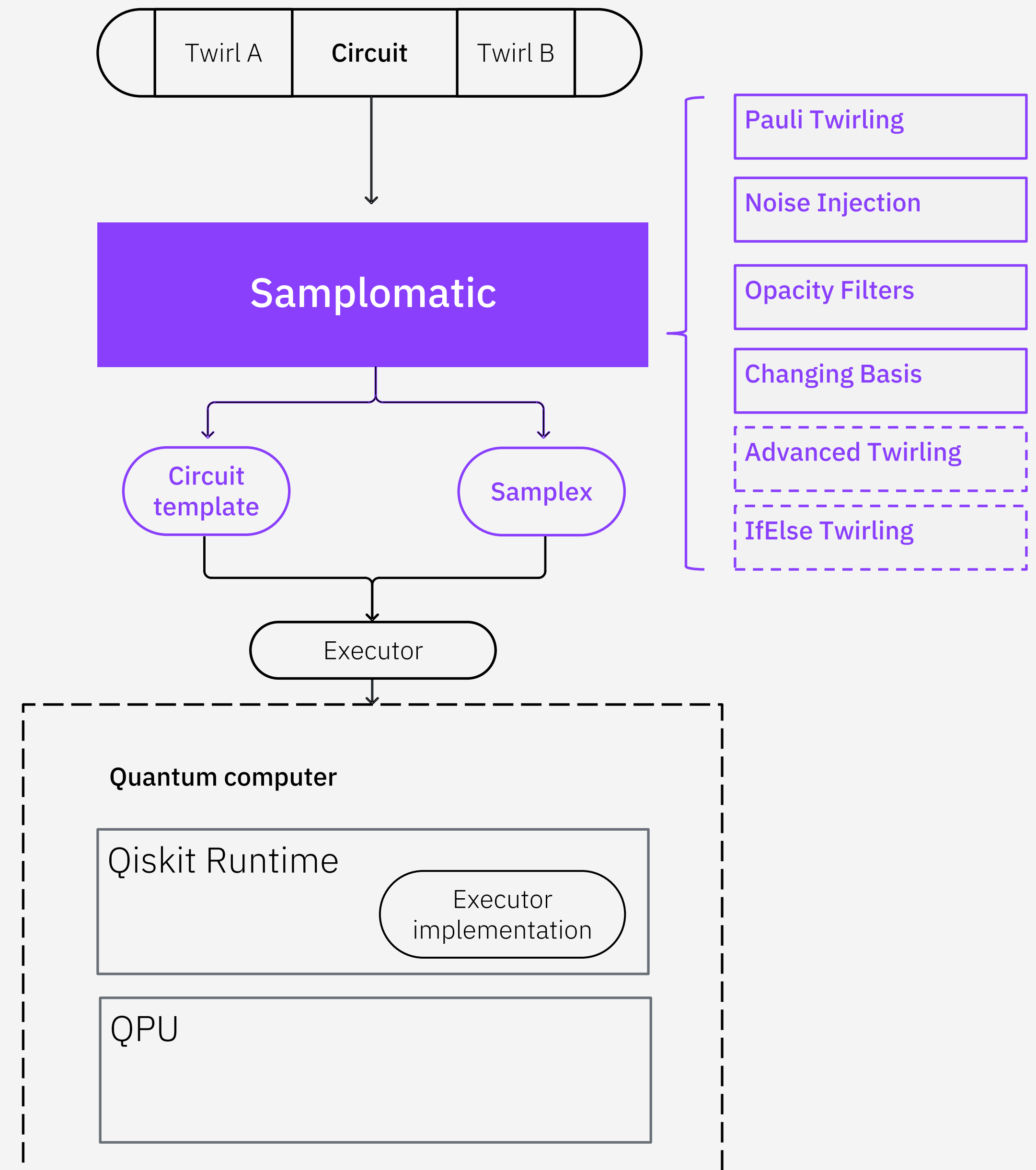


Executor and Samplomatic

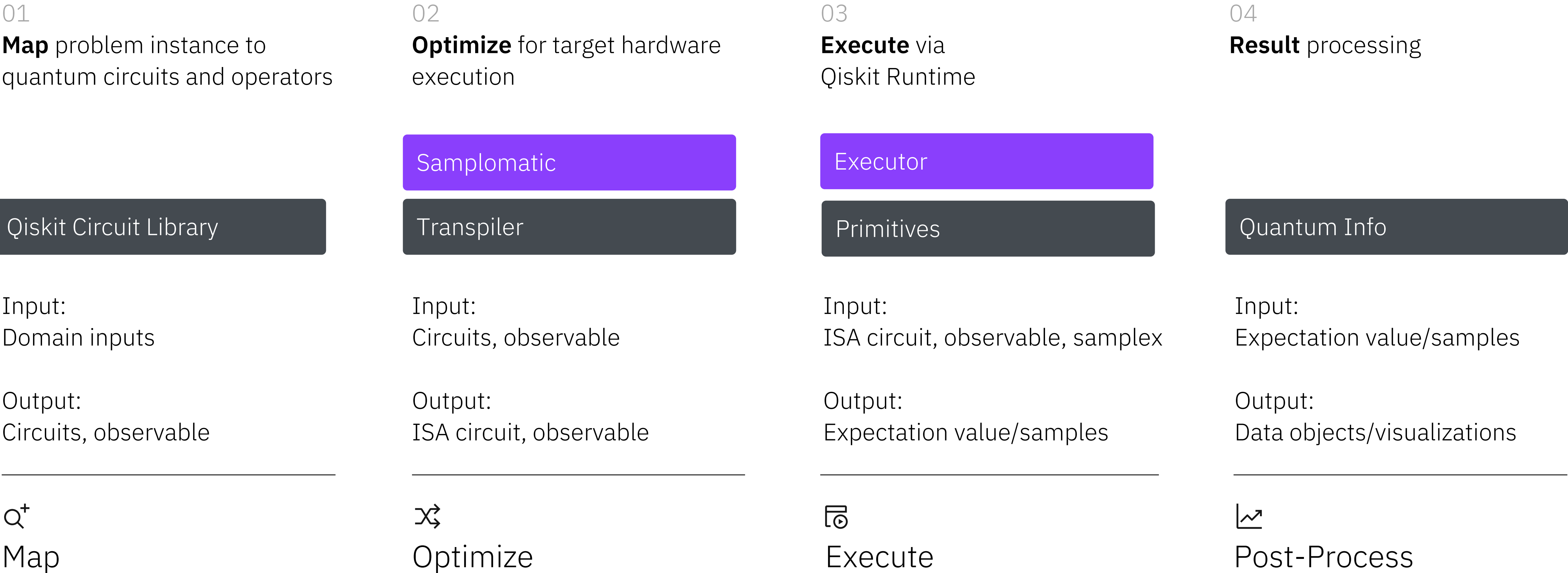
Introducing **Samplomatic** and a new samplex-accepting primitive (Executor)

Extended flexibility in circuit layering and twirling for enhanced error suppression and mitigation

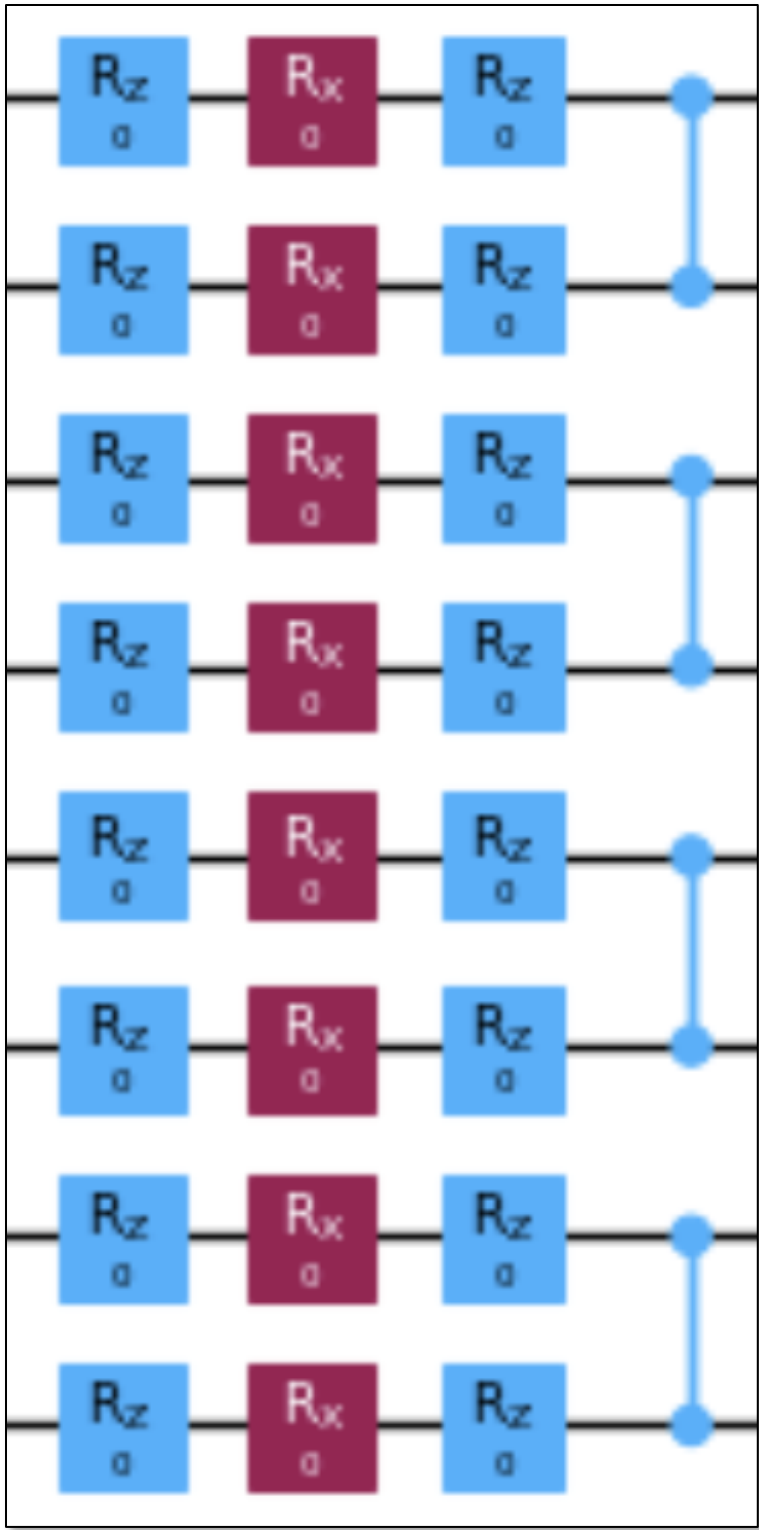
1. Insert different types of annotations into circuits.
2. Transform circuit into template circuit and sampling graph (**samplex**)
3. Send template and samplex as input into the executor primitive



Samplomatic within the Qiskit pattern structure



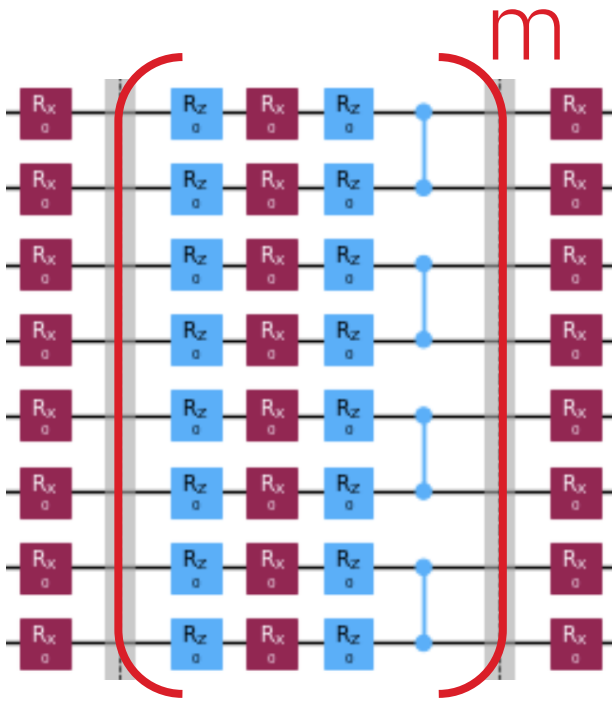
Dressed Layers for Portability



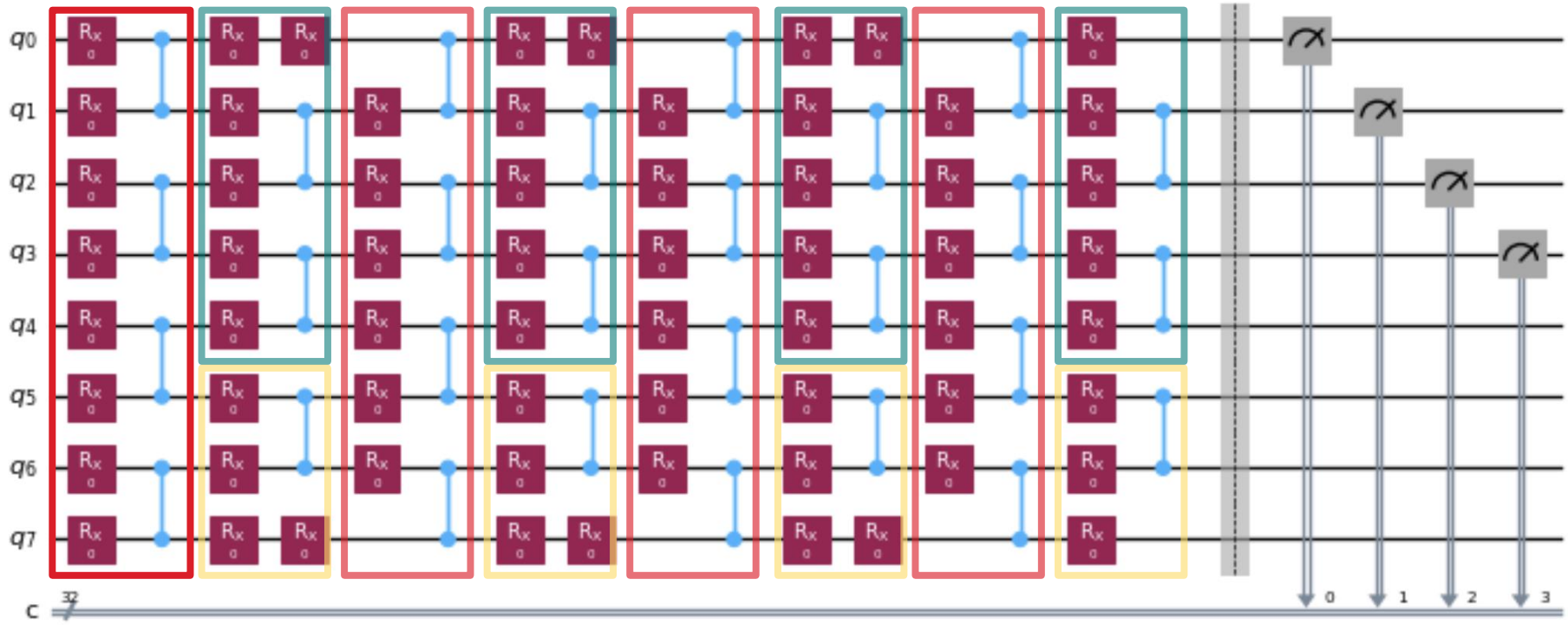
dressing

user gates ○ random gates

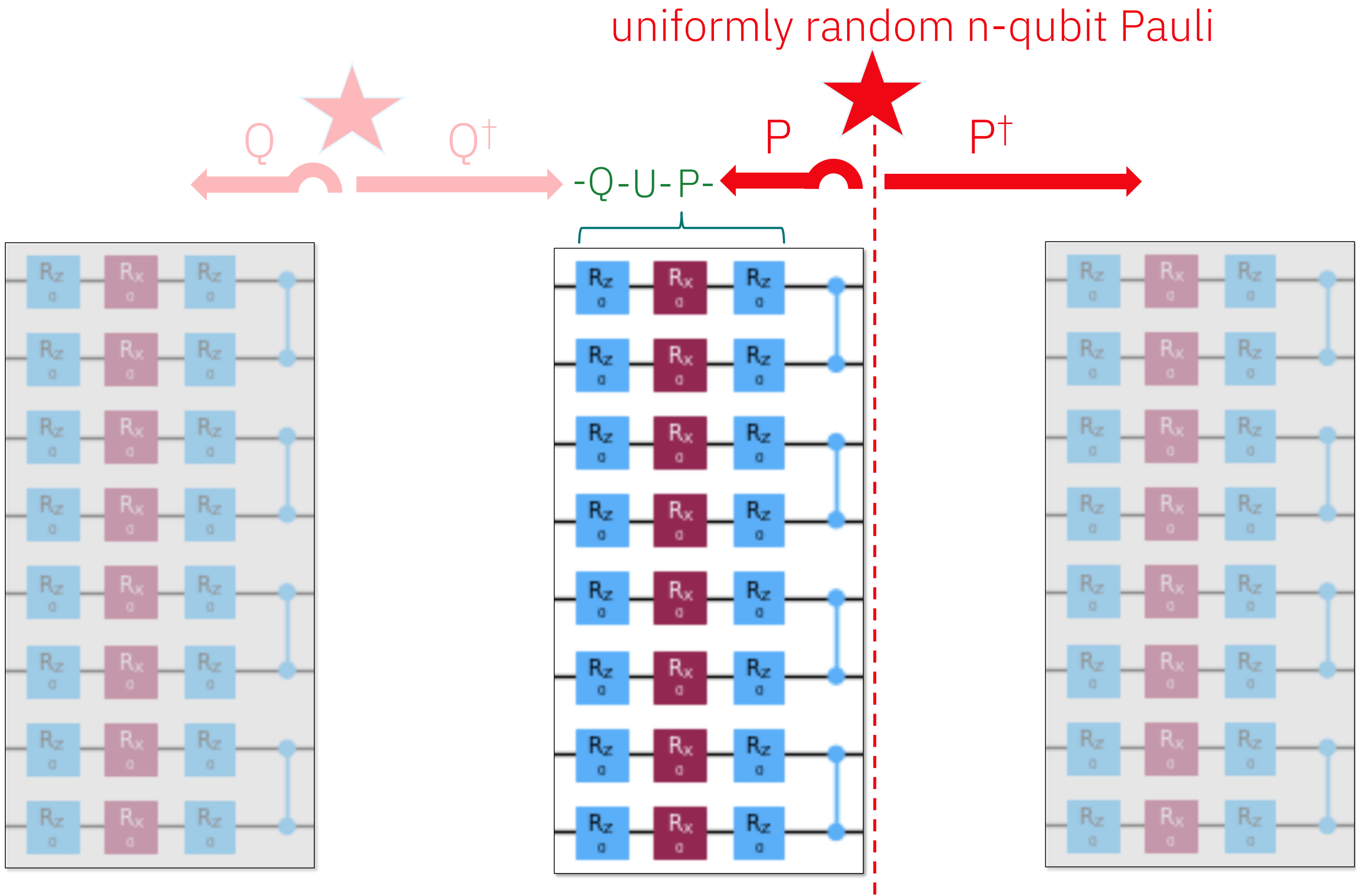
Noise Learning



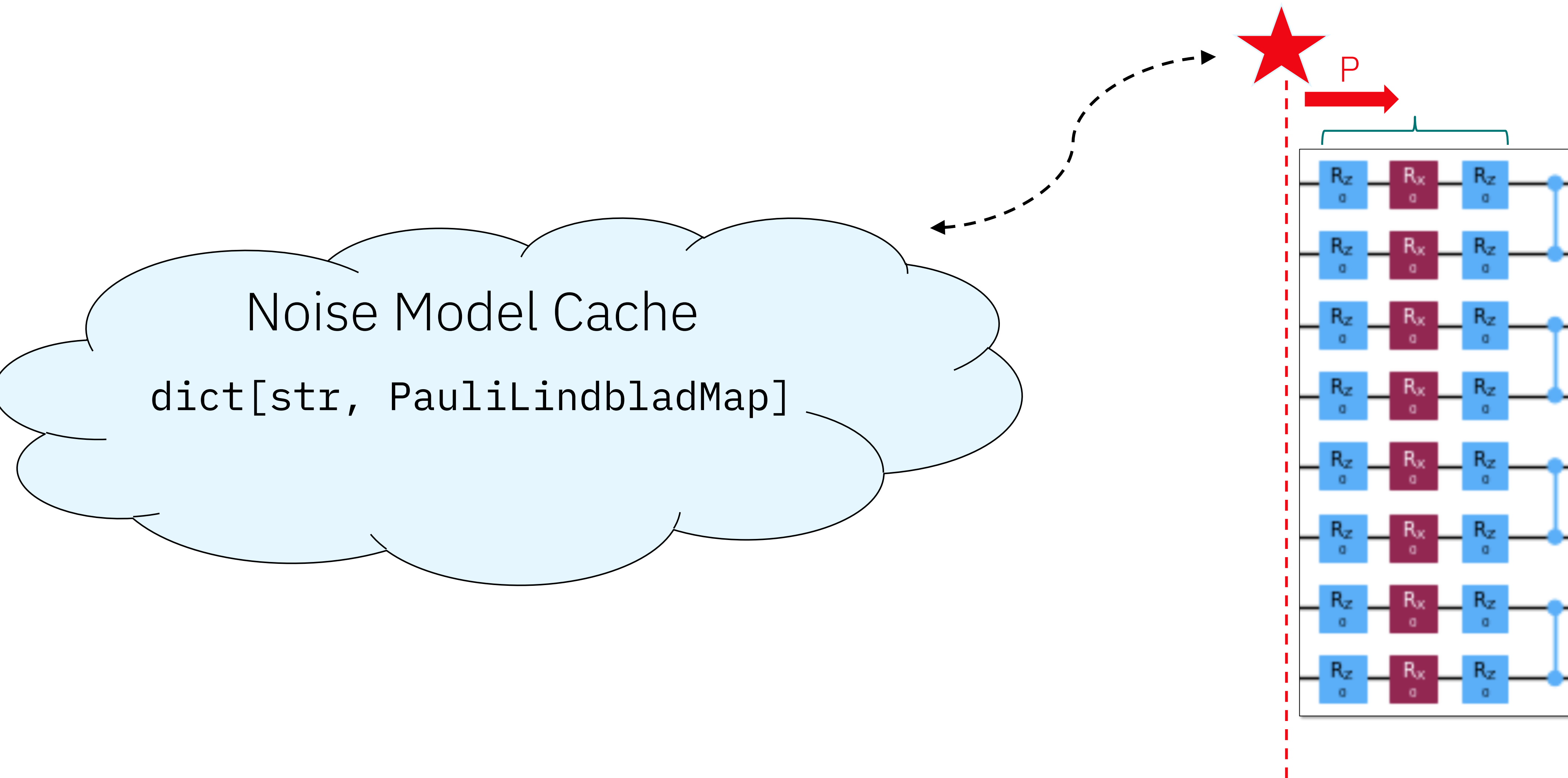
Application Circuit



Examples of Dressing Content: Pauli Twirling

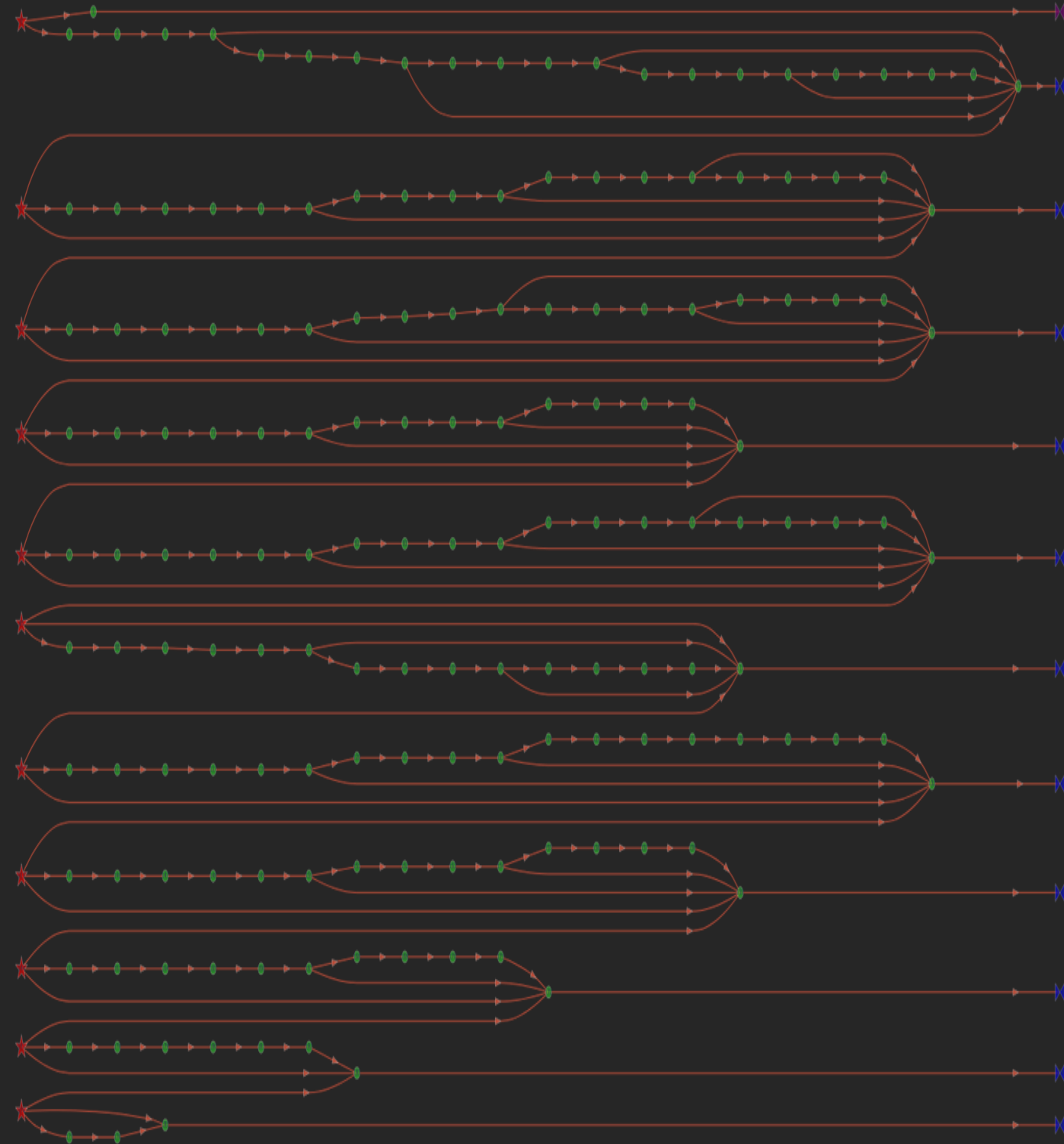


Examples of Dressing Content: Noise Injection



Workflow

1. ***Declare*** intent with samplomatic's box annotation directives.
2. ***Learn*** your noise models.
3. ***Build*** your randomization procedure locally.
 - 3.1 **Inspect** the results of the build.
4. ***Execute*** your application.
5. ***Process*** your results.



0. Installing Samplomatic

```
%pip install samplomatic
```

```
import samplomatic
```

- New, open-source, vendor-agnostic Python library
 - <https://github.com/Qiskit/samplomatic>
- Extensible, performant, general framework for circuit randomization
 - More tools to come
- We want your feedback and ideas!
 - Let's build modular error mitigation tooling together

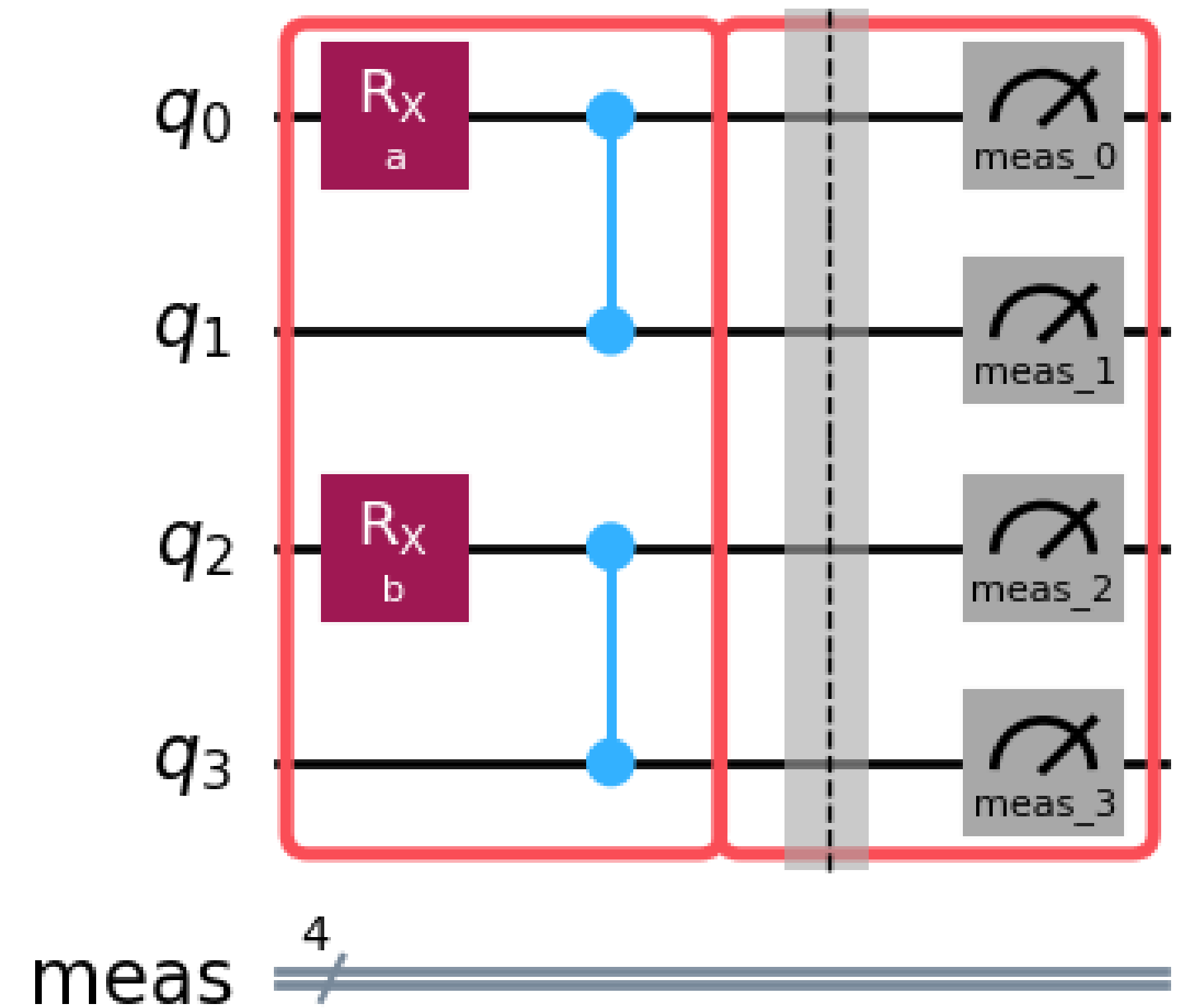
1. Declare *Annotated Boxes*

```
from samplomatic import Twirl
```

```
circuit = QuantumCircuit(4)
```

```
with circuit.box([Twirl()]):  
    circuit.rx(Parameter("a"), 0)  
    circuit.rx(Parameter("b"), 2)  
    circuit.cz(0, 1)  
    circuit.cz(2, 3)
```

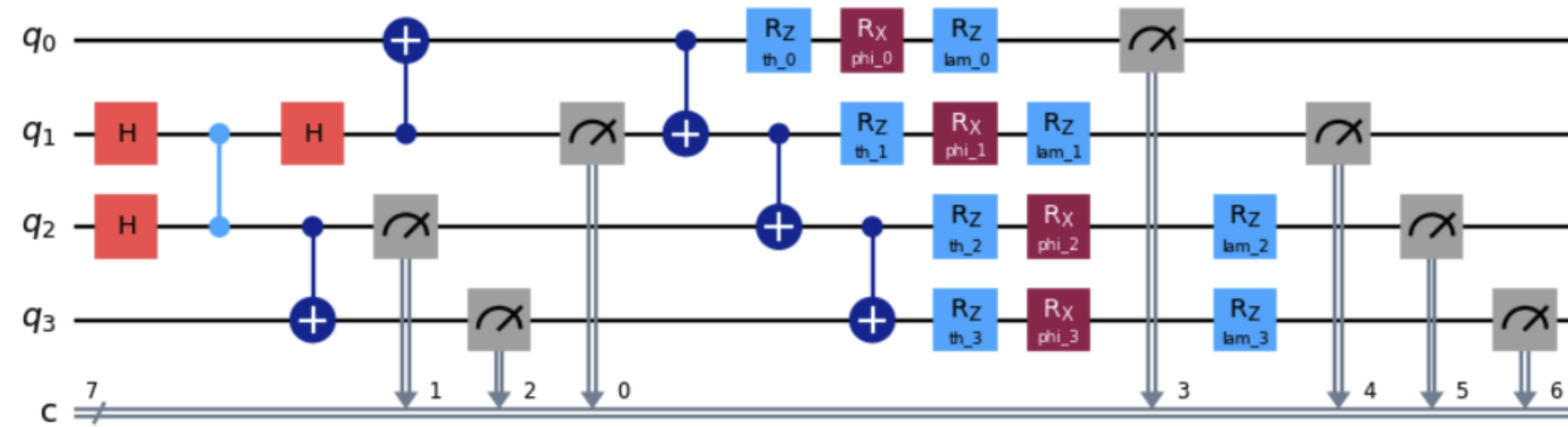
```
with circuit.box([Twirl()]):  
    circuit.measure_all()
```



1. Declare *Annotated Boxes*

Function	Parameter Name	Parameter Value	Effect
Twirl	group	‘pauli’	Declare the box should be twirled.
	dressing	‘left’, ‘right’	
	decomposition	‘rzrx’, ‘rzsx’	
InjectNoise	ref	str	Declare the box should have noise injected from a noise model.
	modifier_ref	str	
ChangeBasis	ref	str	Declare the box should insert gates to enter or leave a basis.
	mode	‘prepare’, ‘measure’	
	decomposition	‘rzrx’, ‘rzsx’	

1. Declare *Annotated Boxes*

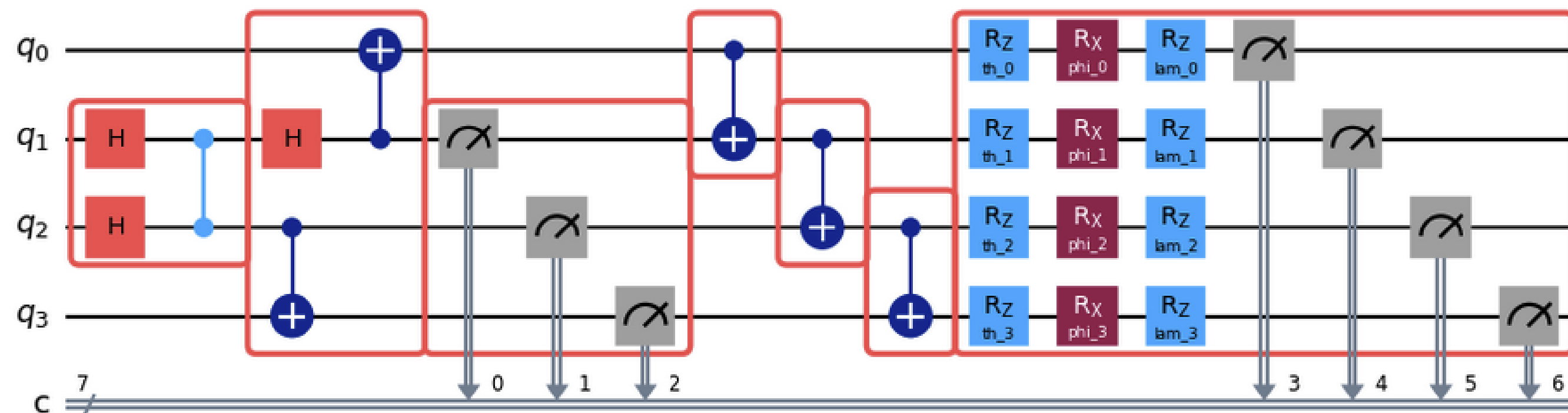


samplomatic.transpiler.generate_boxing_pass_manager(...)

```
from samplomatic.transpiler import generate_boxing_pass_manager

boxing_pass_manager = generate_boxing_pass_manager(
    enable_gates=True,
    enable_measures=True,
)

transpiled_circuit = boxing_pass_manager.run(circuit)
transpiled_circuit.draw("mpl", scale=0.8)
```



2. Learn Noise: What's new

Legacy NoiseLearner

Inputs.

- A quantum circuit.
- Options.

Outputs.

- A sequence of noise models (one per unique instruction).

Learning process.

- Break the circuits into layers and identify the unique ones.
- Learn the noise of the unique layers.

Limitations.

- The process of finding the unique layers happen in the server. It is opaque and sometimes counter-intuitive.
- It only returns the noise of gate layers, not of measurement layers.

NoiseLearnerV3

Inputs.

- A sequence of boxes. Each box contains one layer of operations.
- Options, such as number of shots, number of randomizations, ...

Outputs.

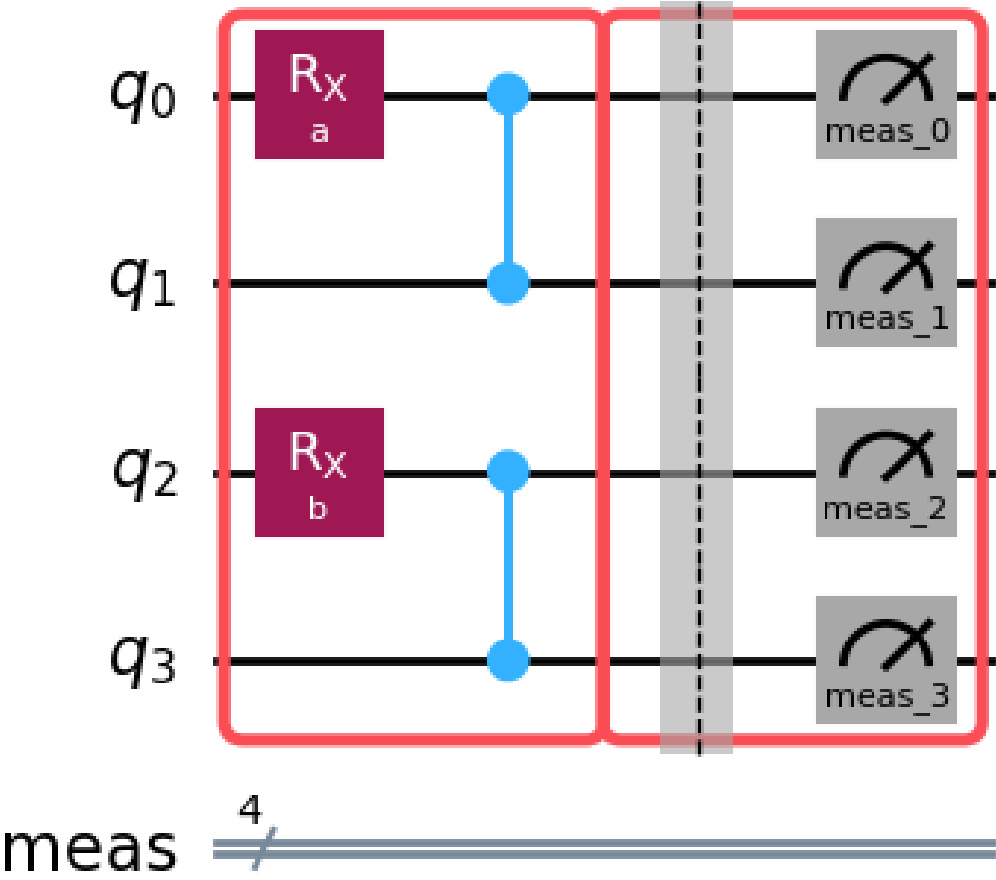
- A sequence of noise models, one per box.
- Connections to `qiskit.quantum_info.PauliLindbladMap`

Learning process.

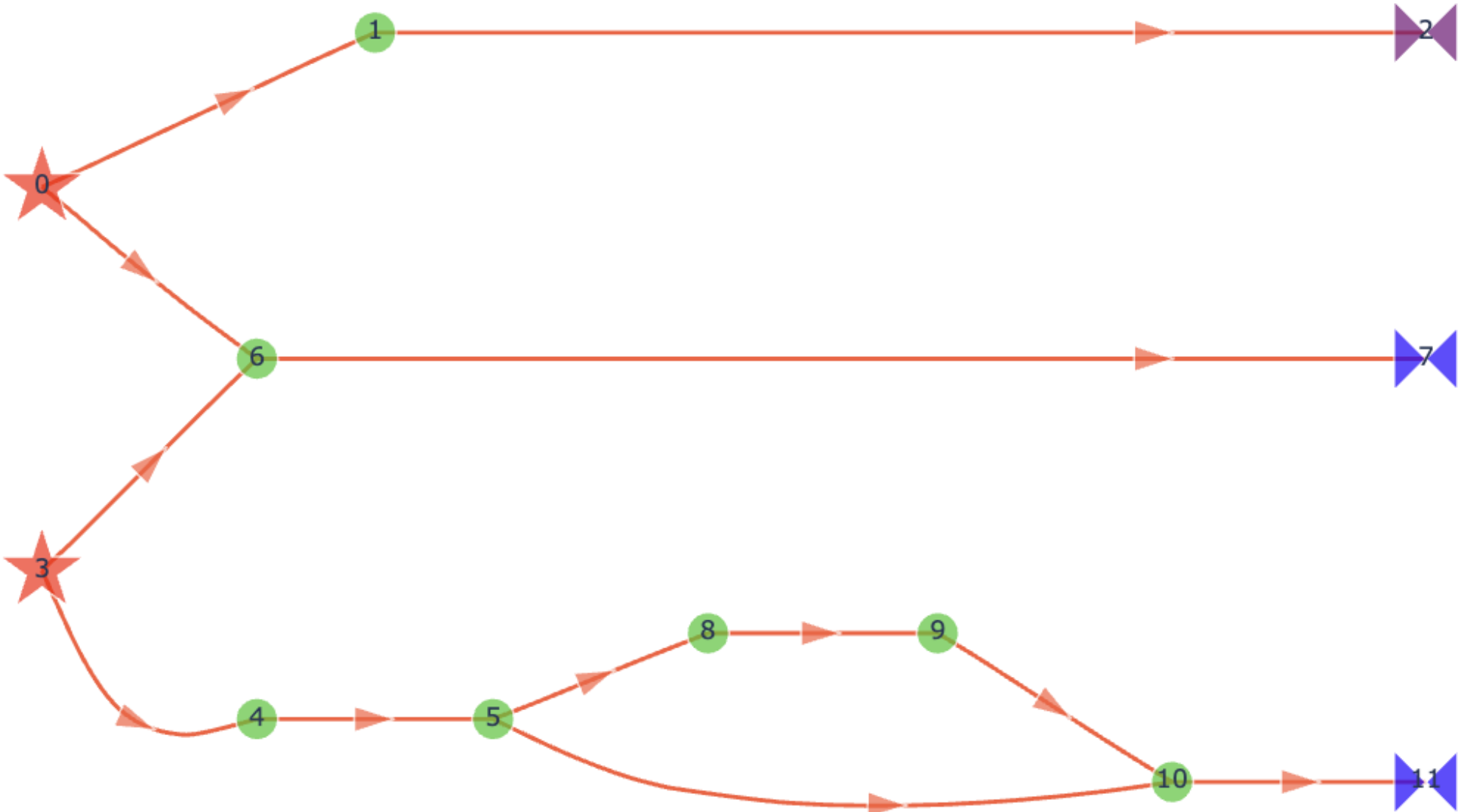
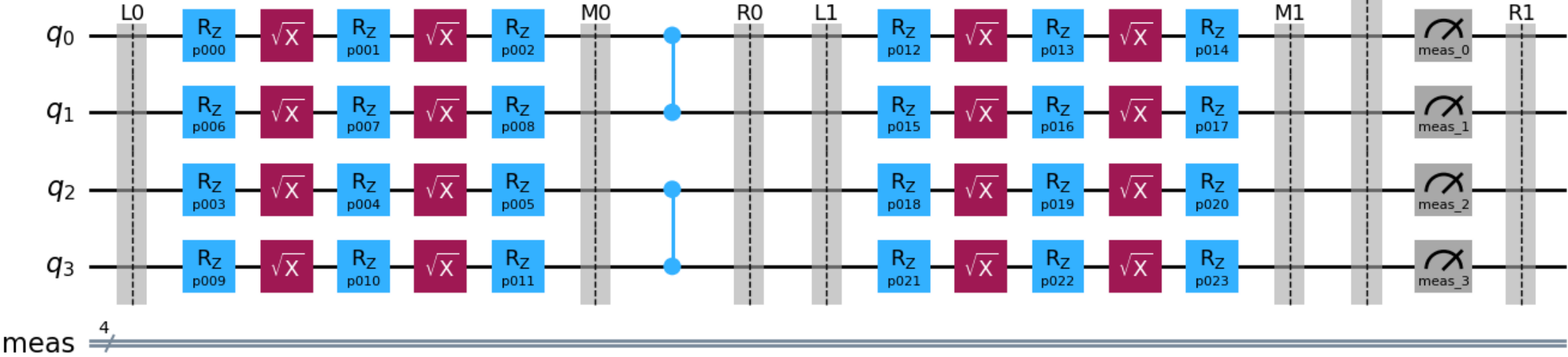
- If an instruction contains a layer of entanglers, do **Pauli Lindblad**.
- If an instruction contains a layer of measurements, do **TREX**.

3. Build

```
from samplomatic import build
template, samplex = build(circuit)
```



build()



template

samplex

3.1 Inspect:

Template circuits: WYSIWYG (What you see is what you get)

```
circuit = QuantumCircuit(4)
```

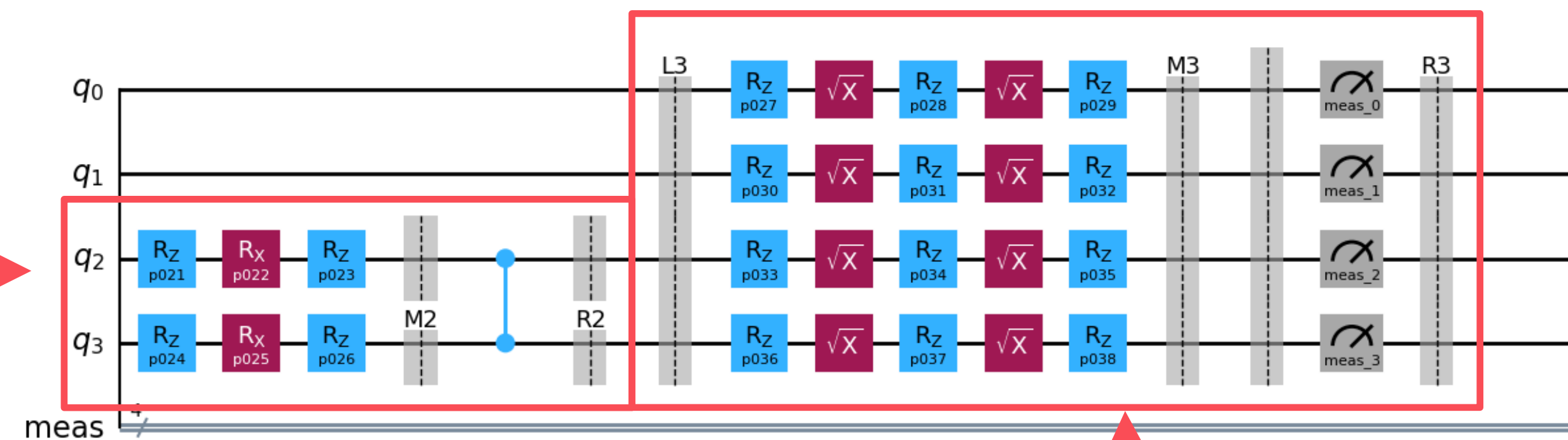
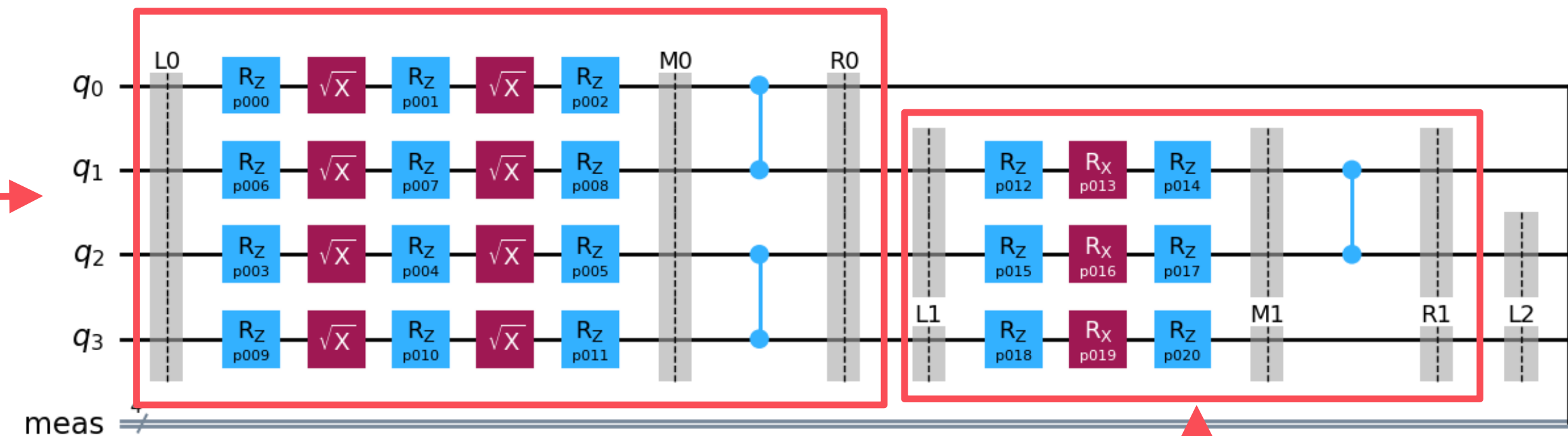
```
with circuit.box([Twirl()]):  
    circuit.rx(Parameter("a"), 0)  
    circuit.rx(Parameter("b"), 2)  
    circuit.cz(0, 1)  
    circuit.cz(2, 3)
```

```
with circuit.box([Twirl(decomposition="rzrx")]):  
    circuit.noop(3)  
    circuit.sx(1)  
    circuit.cz(1, 2)
```

```
with circuit.box([Twirl(decomposition="rzrx")]):  
    circuit.cz(2, 3)
```

```
with circuit.box([Twirl()]):  
    circuit.h(range(4))  
    circuit.measure_all()
```

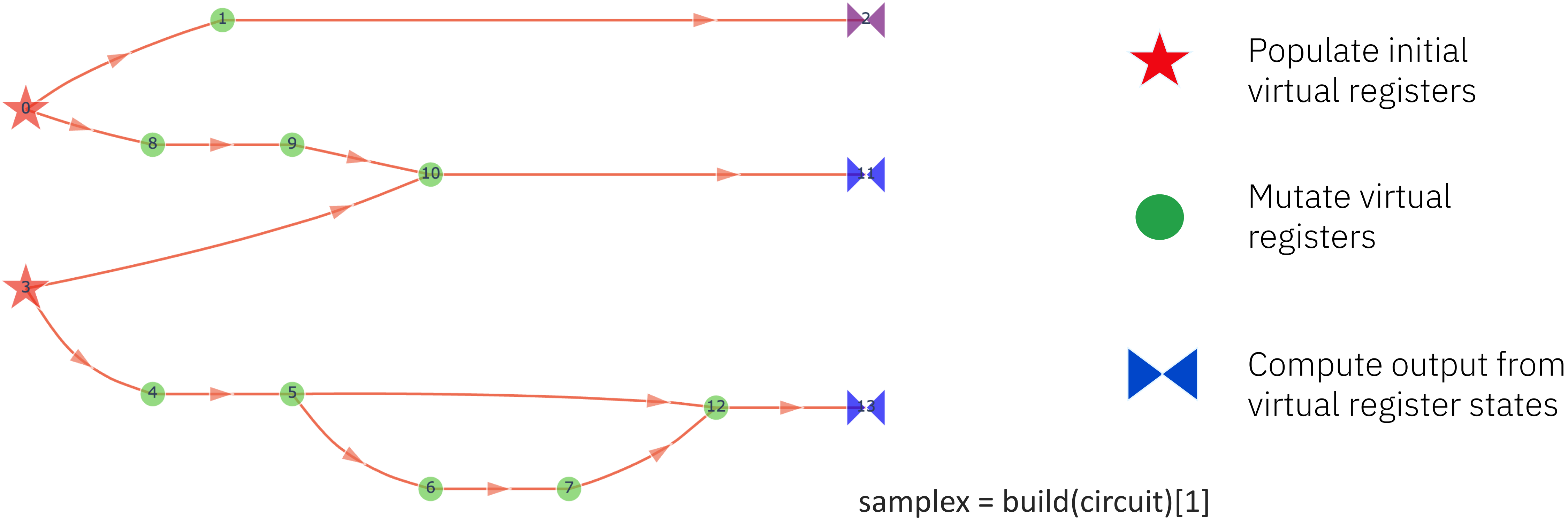
template = build(circuit)[0]



3.1 Inspect:

Template circuits: WYSIWYG

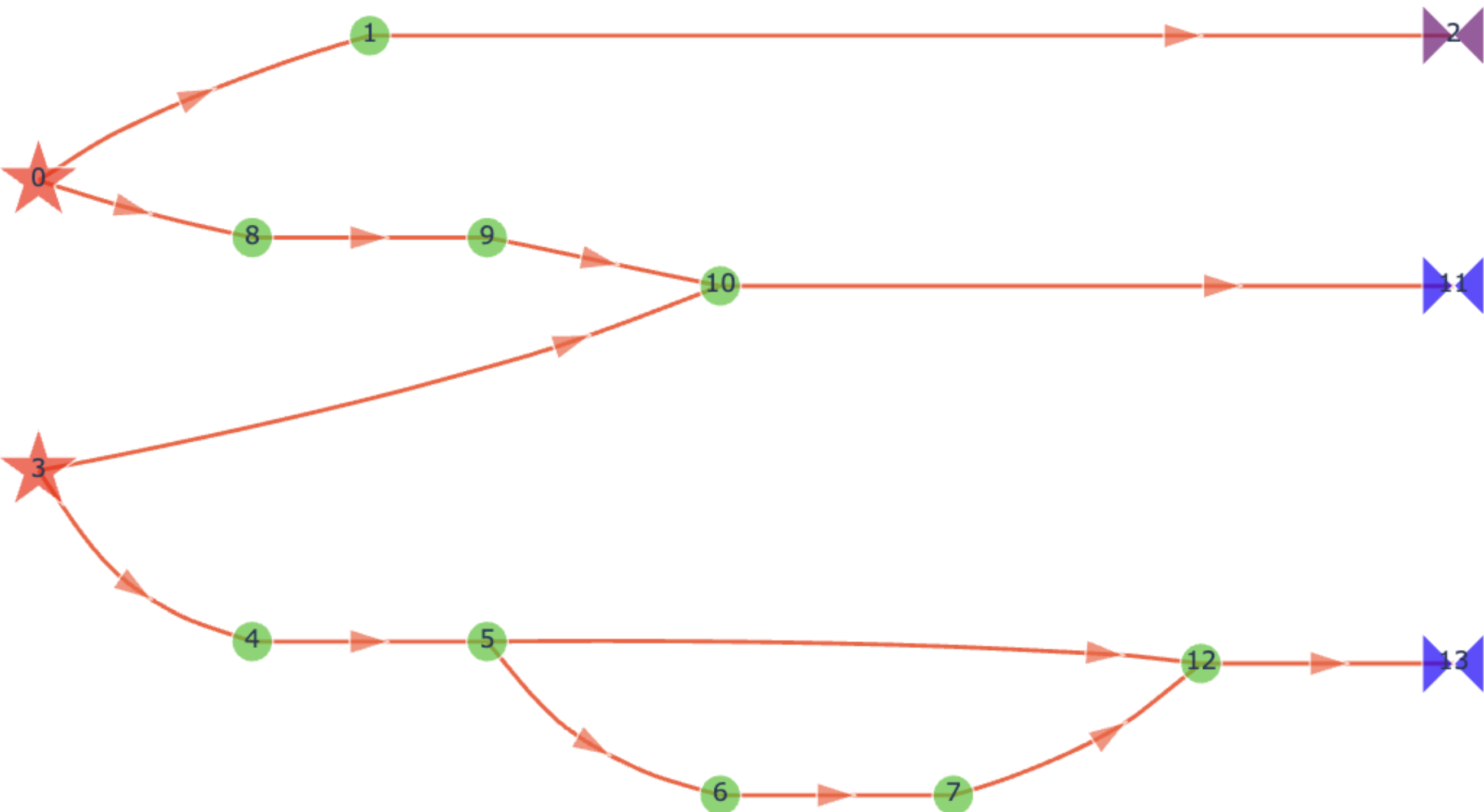
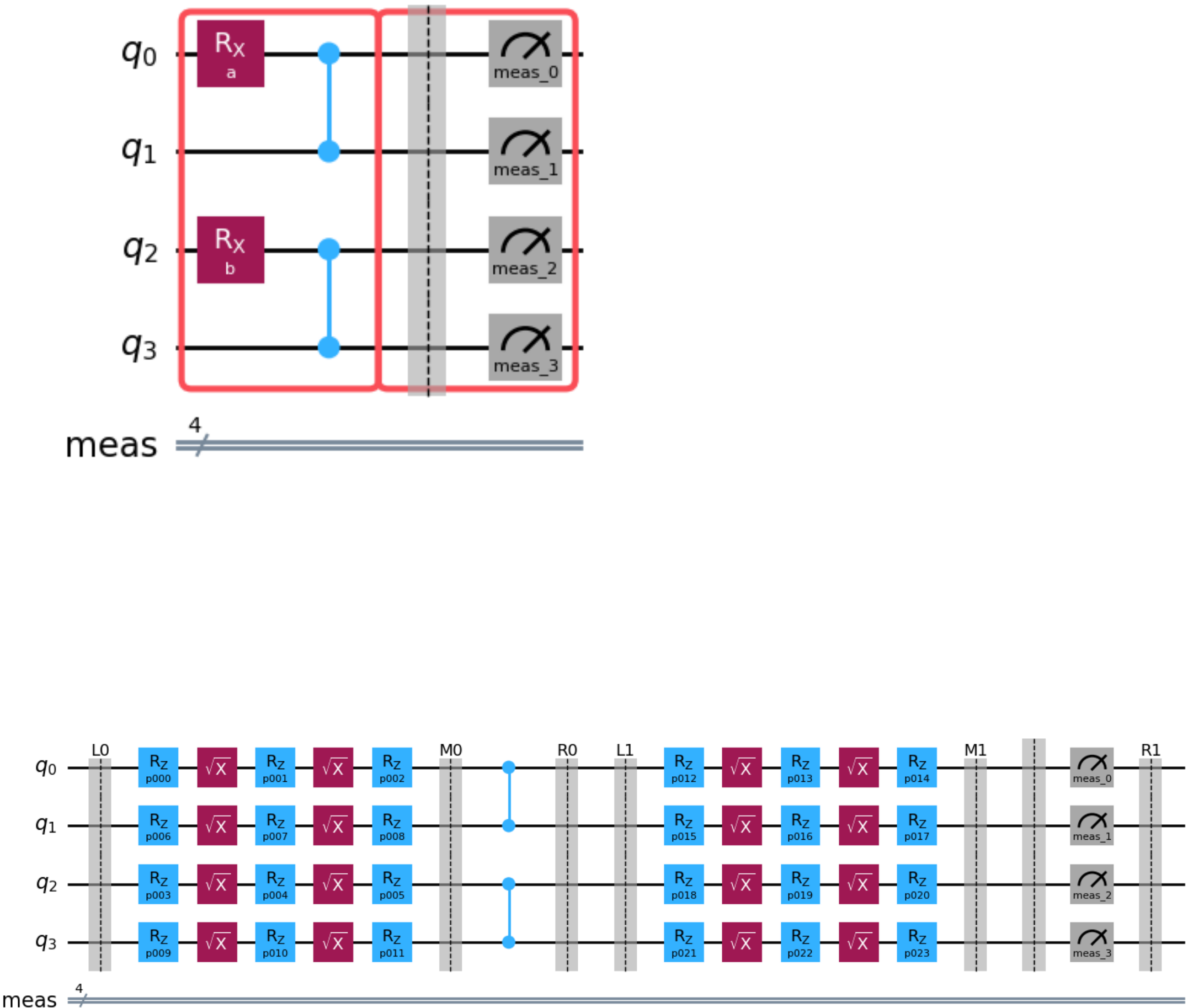
Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

Template circuits: WYSIWYG

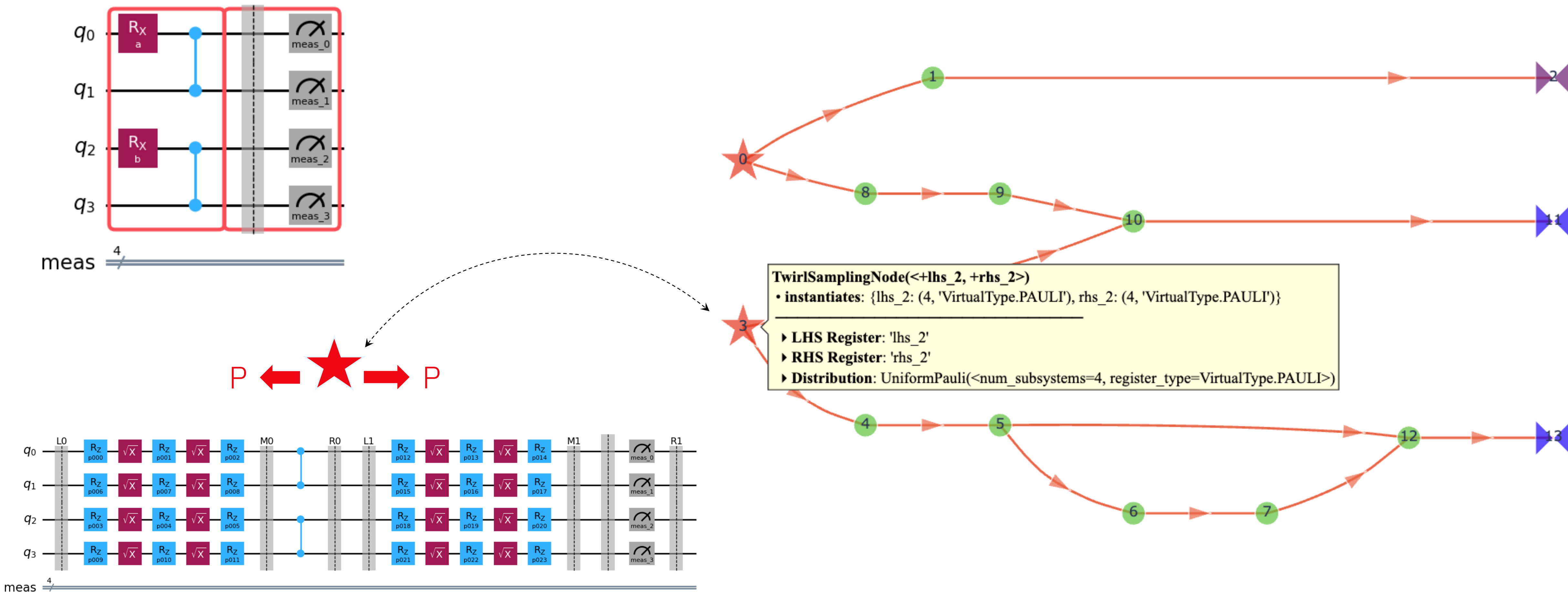
Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

Template circuits: WYSIWYG

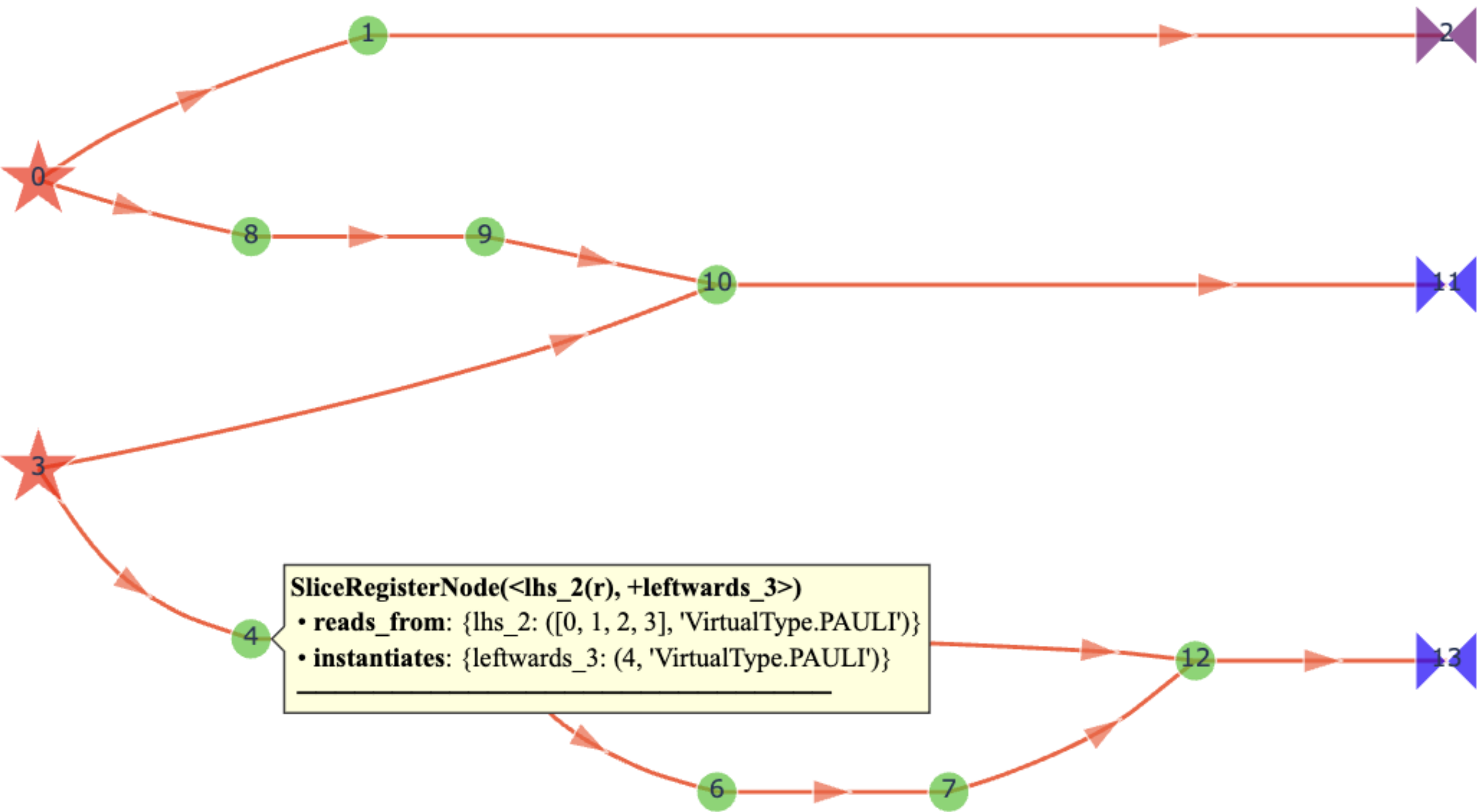
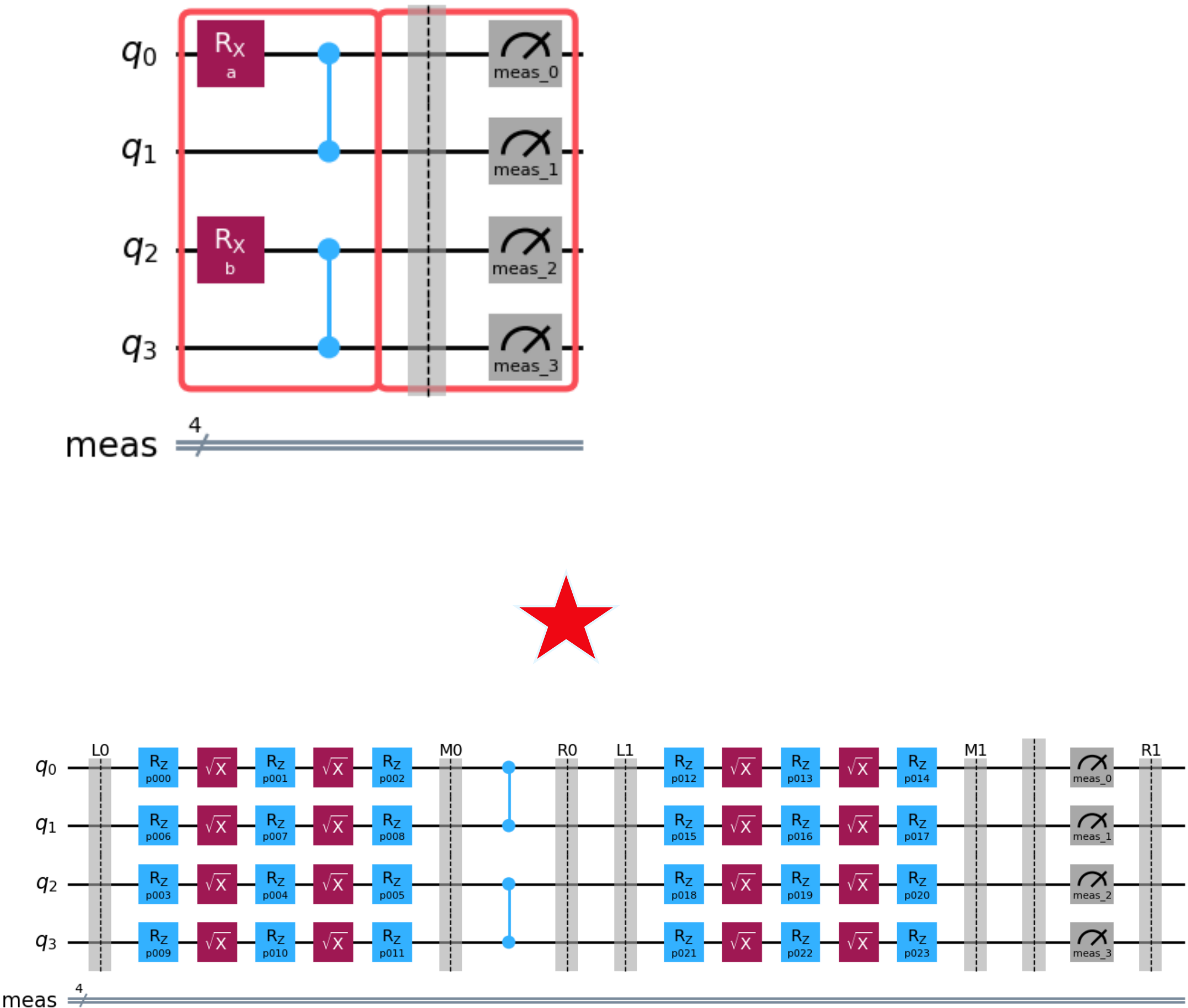
Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

Template circuits: WYSIWYG

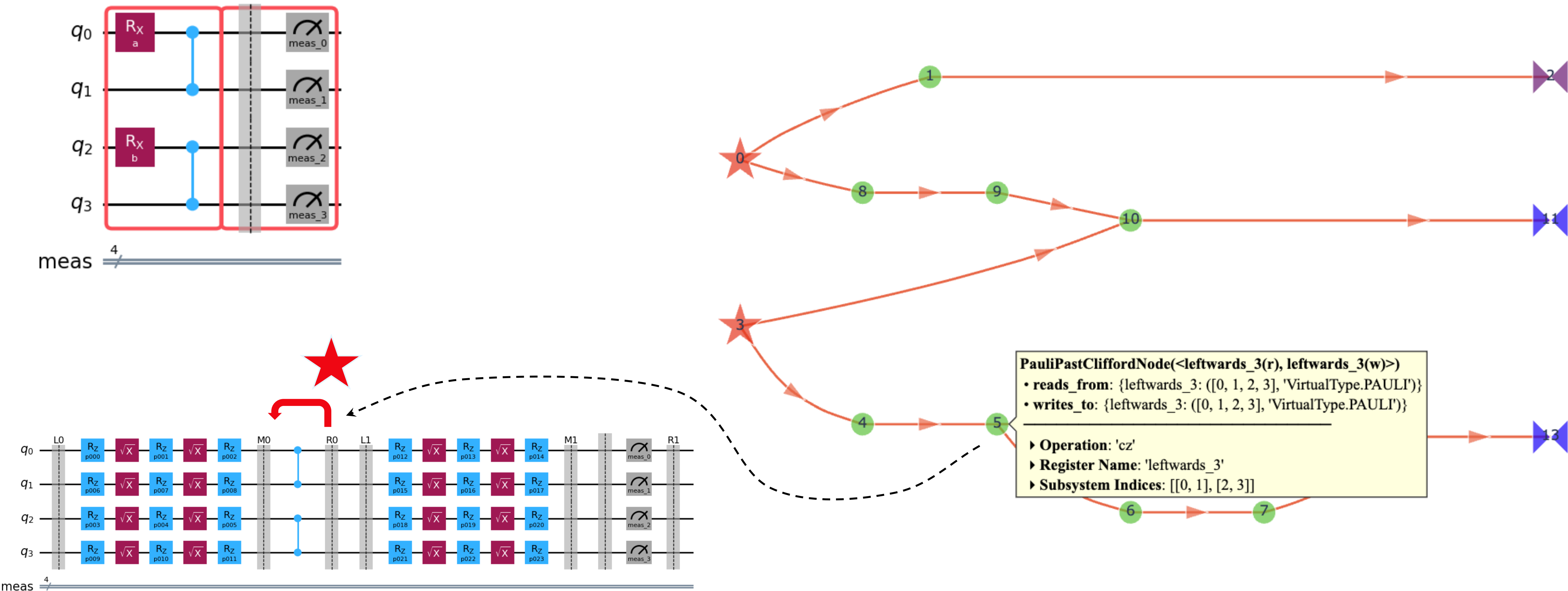
Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

Template circuits: WYSIWYG

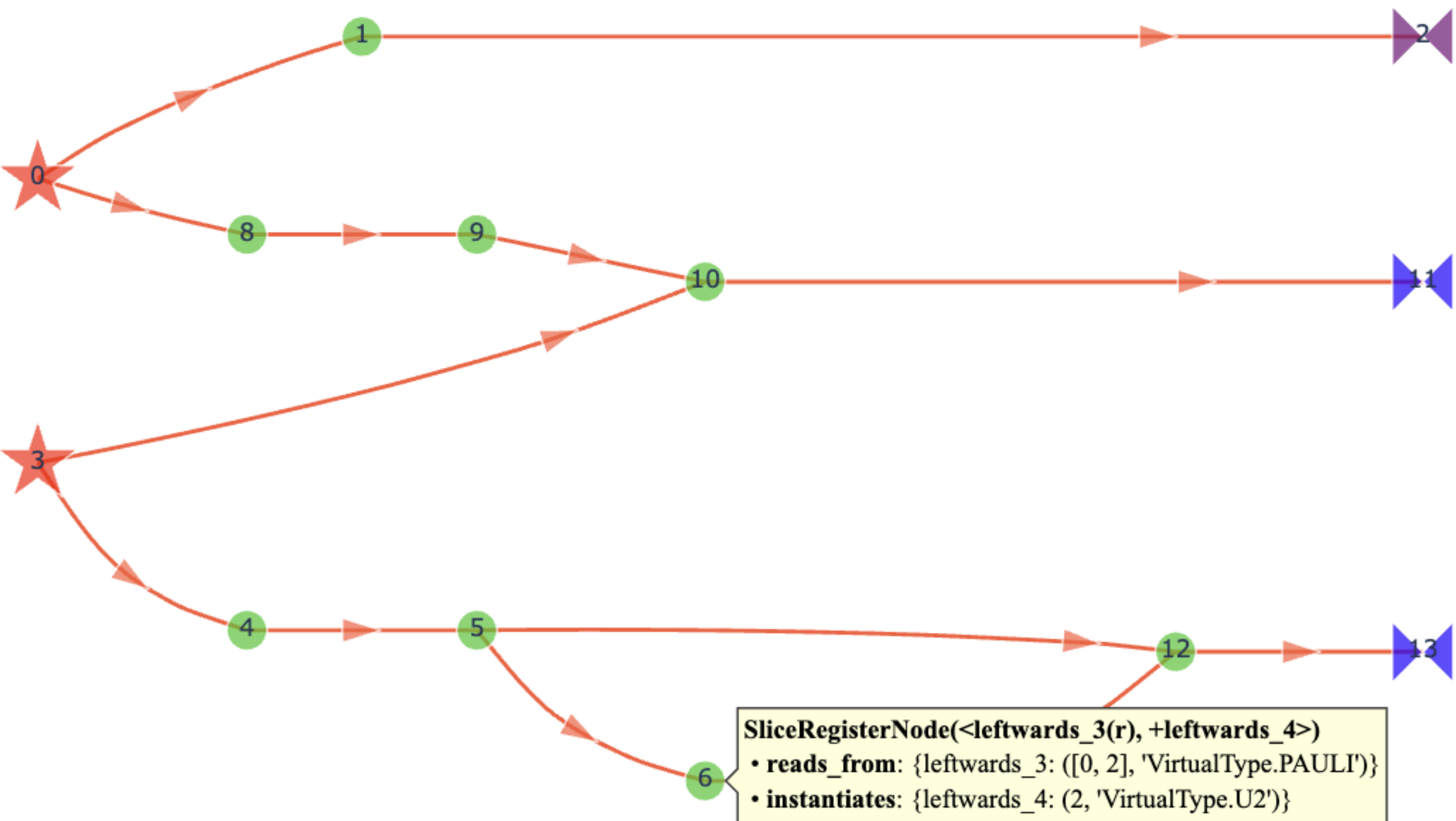
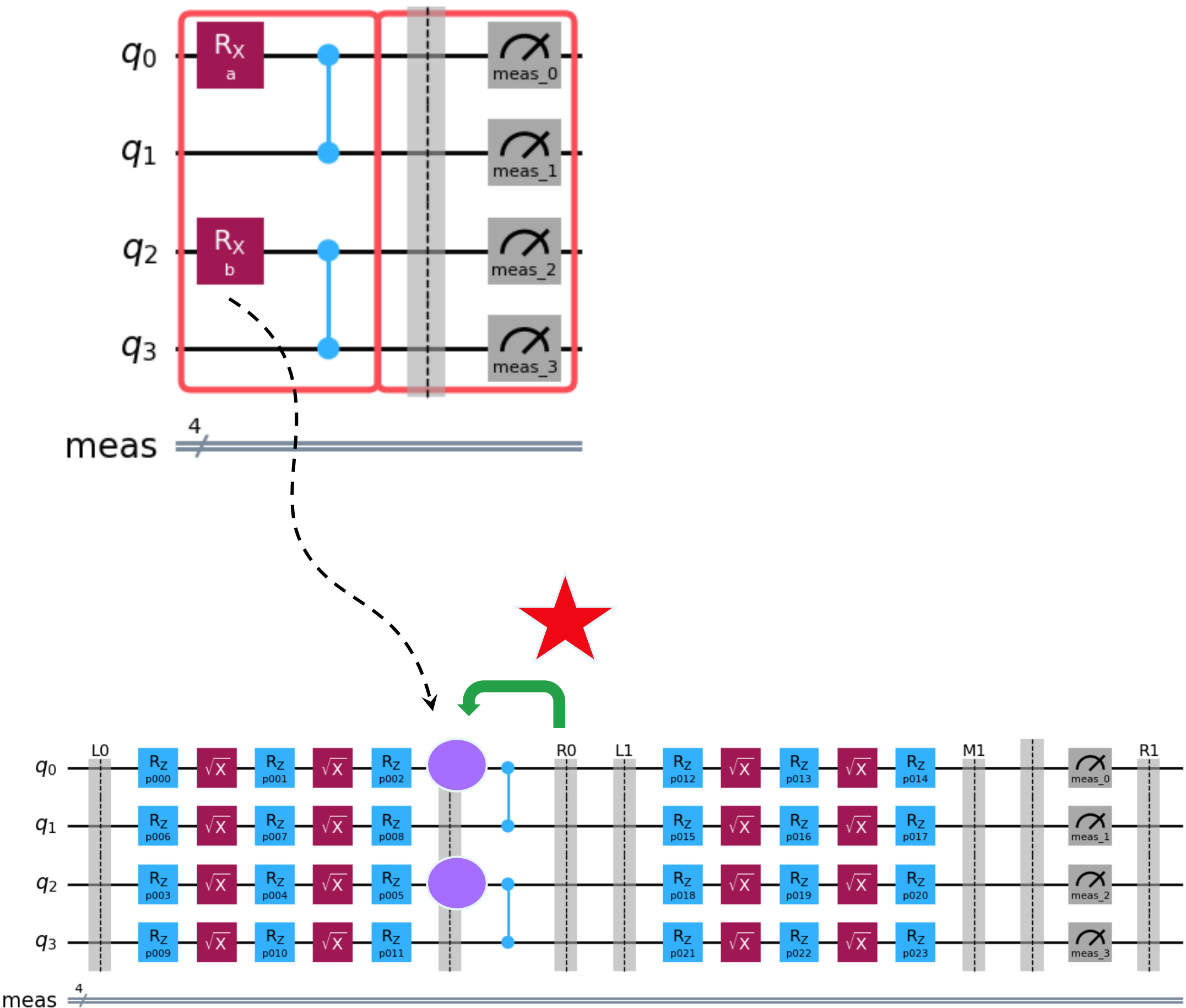
Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

Template circuits: WYSIWYG

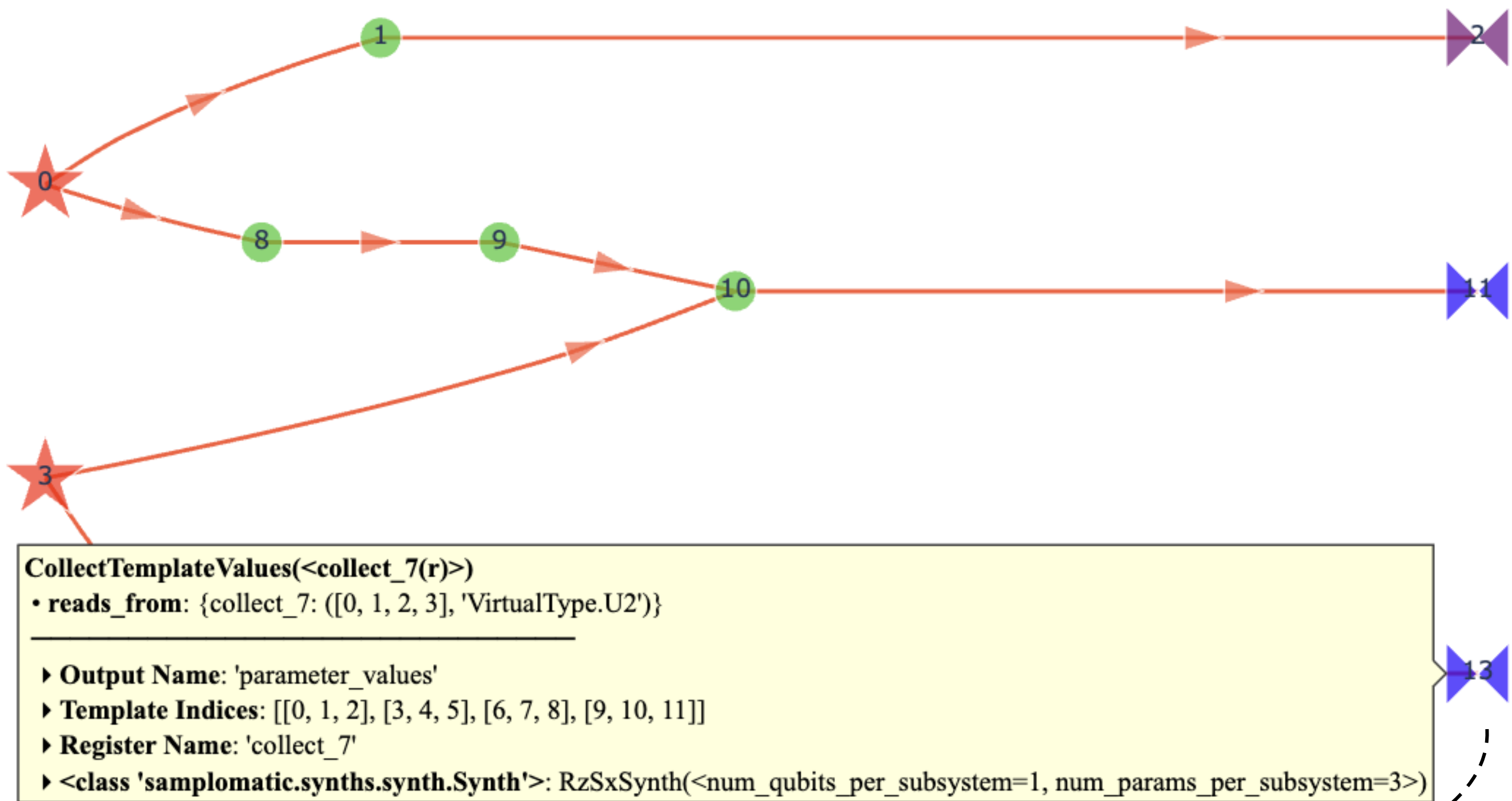
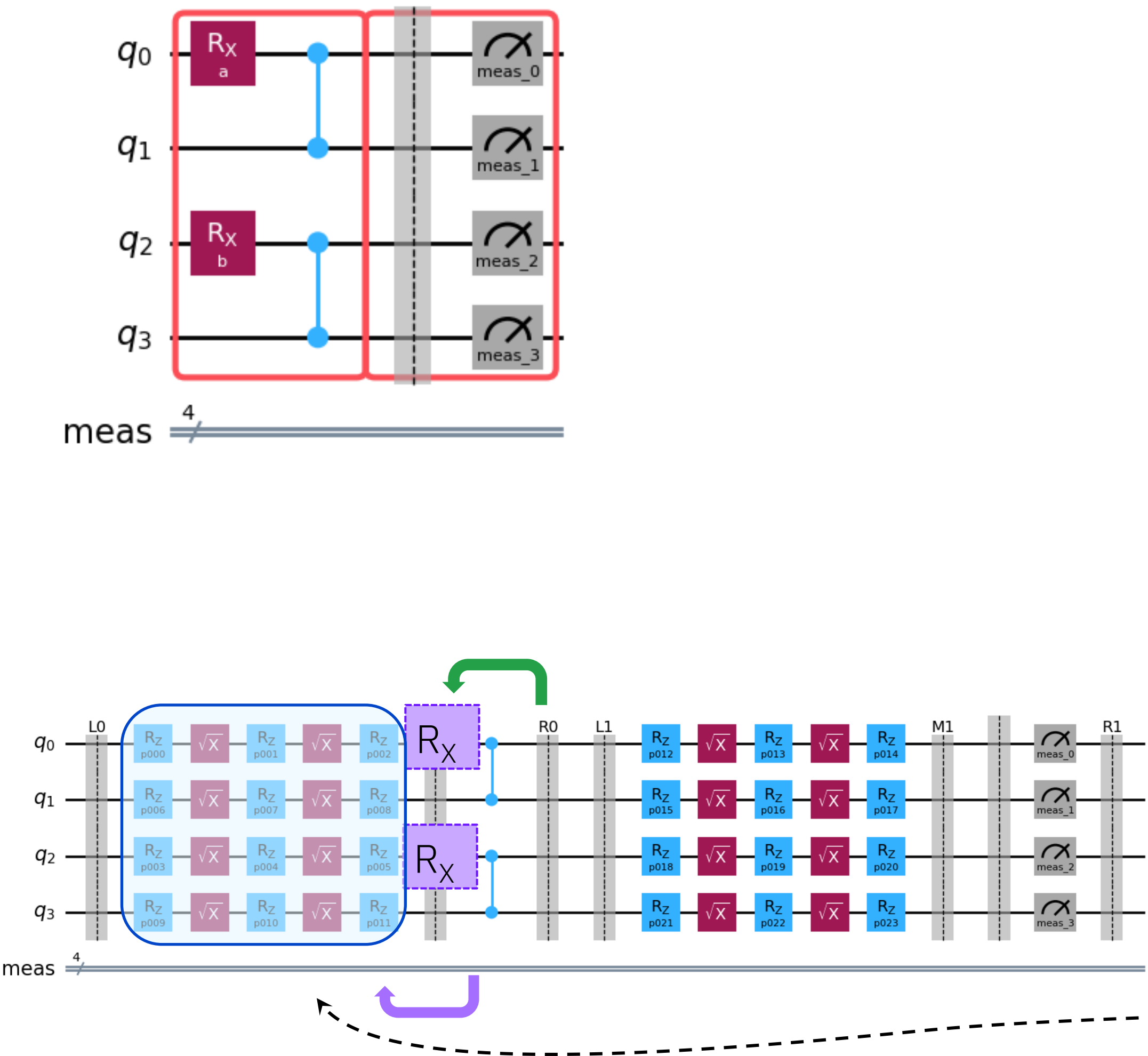
Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

Template circuits: WYSIWYG

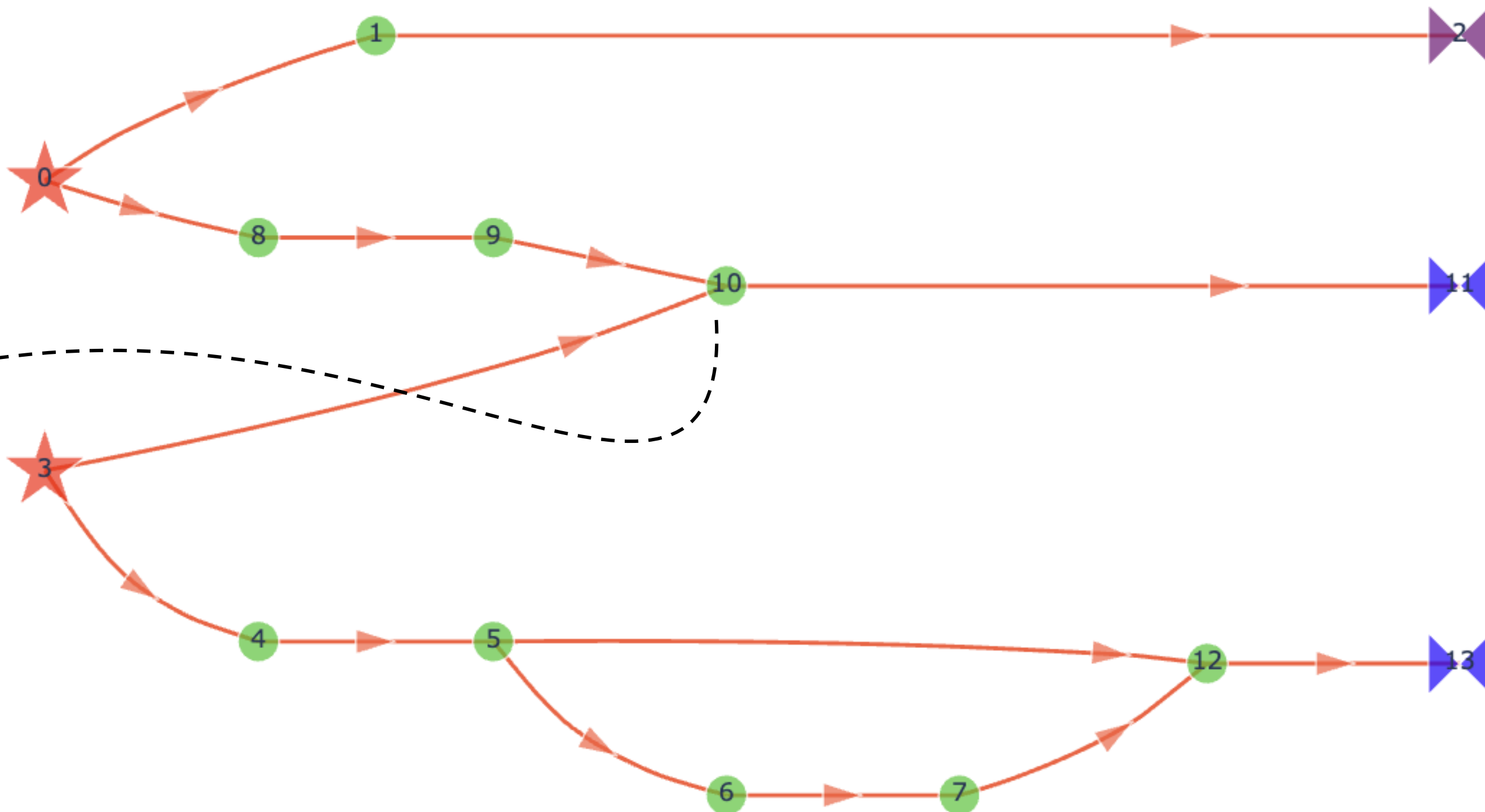
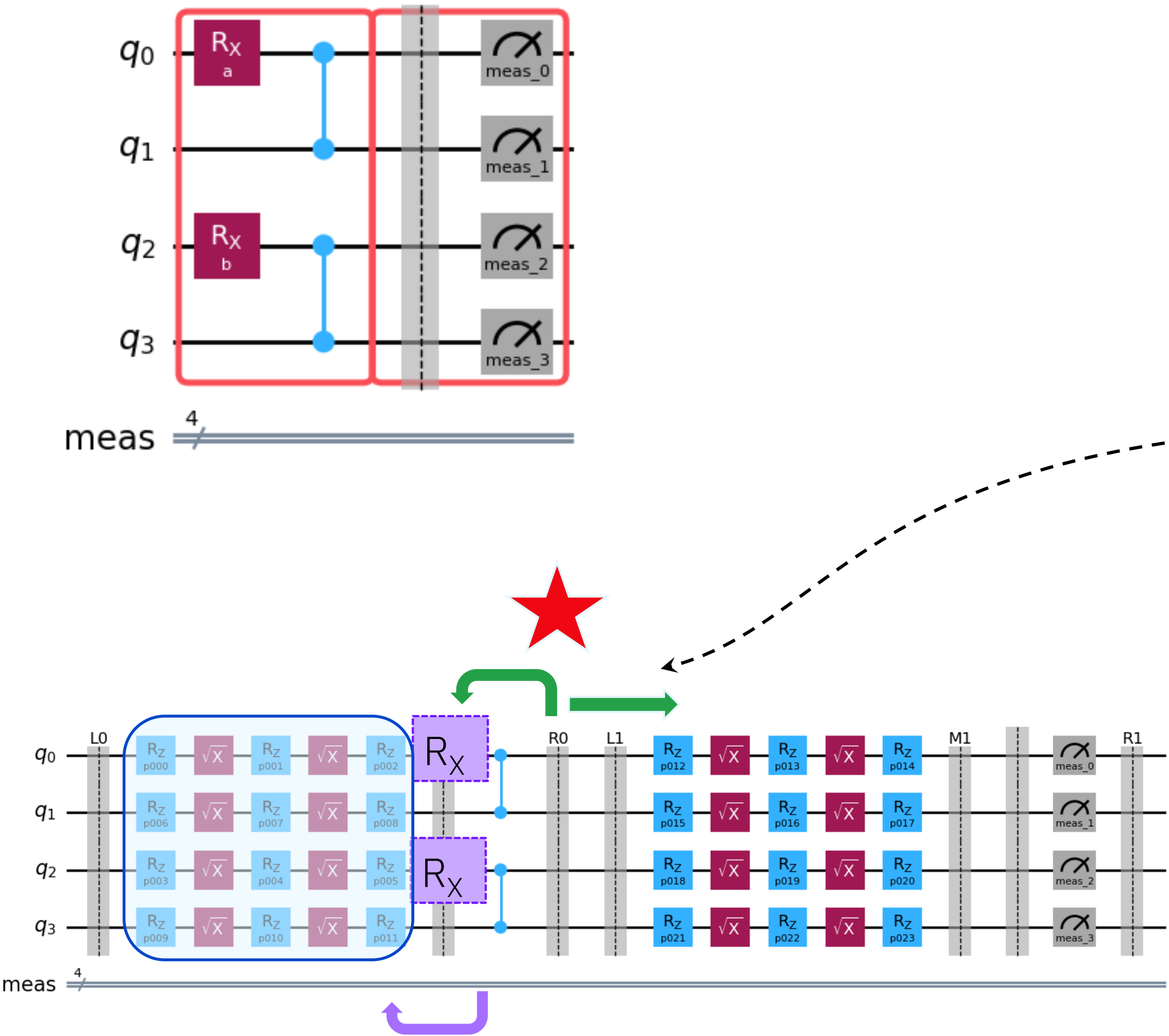
Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

Template circuits: WYSIWYG

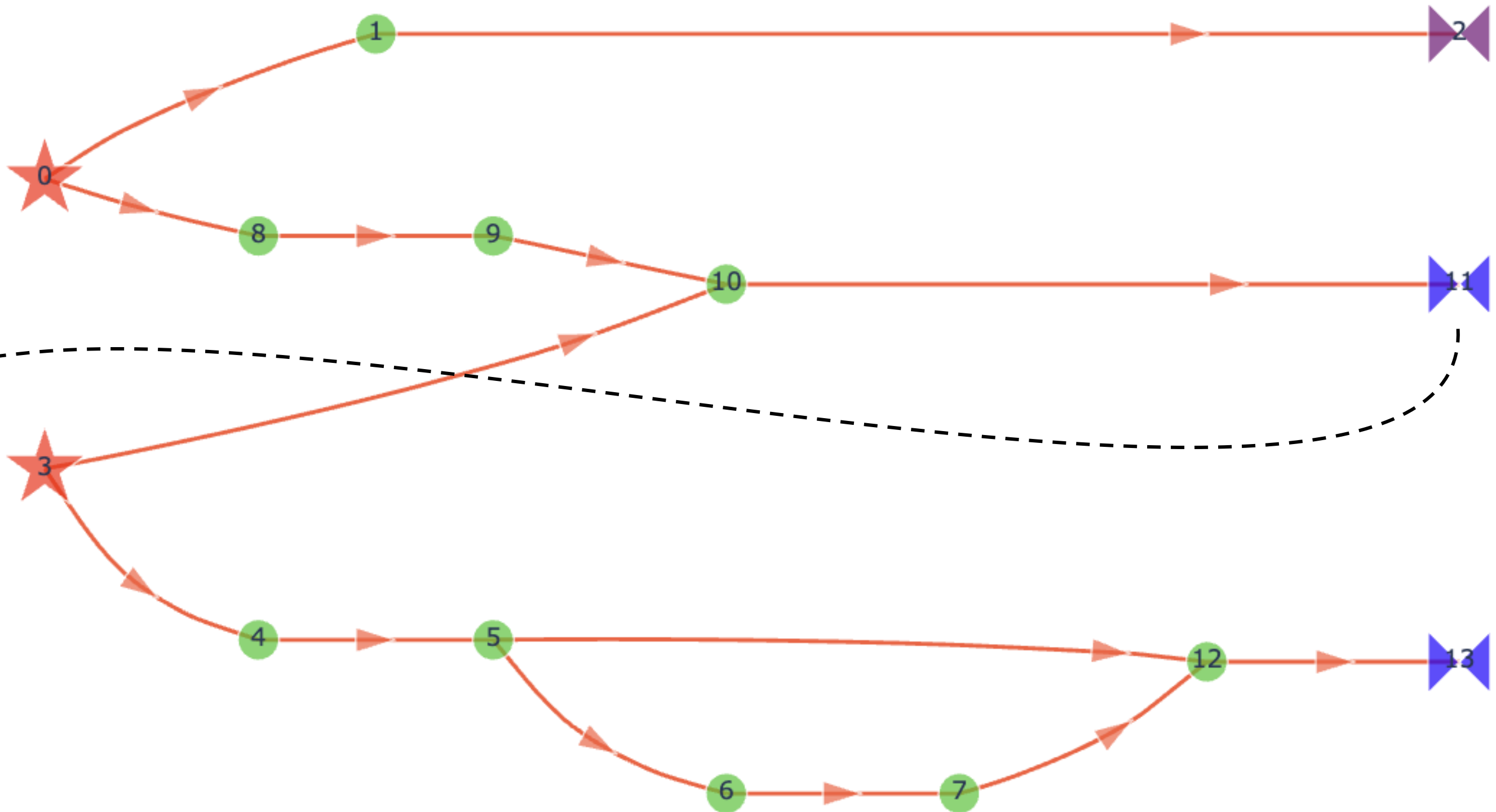
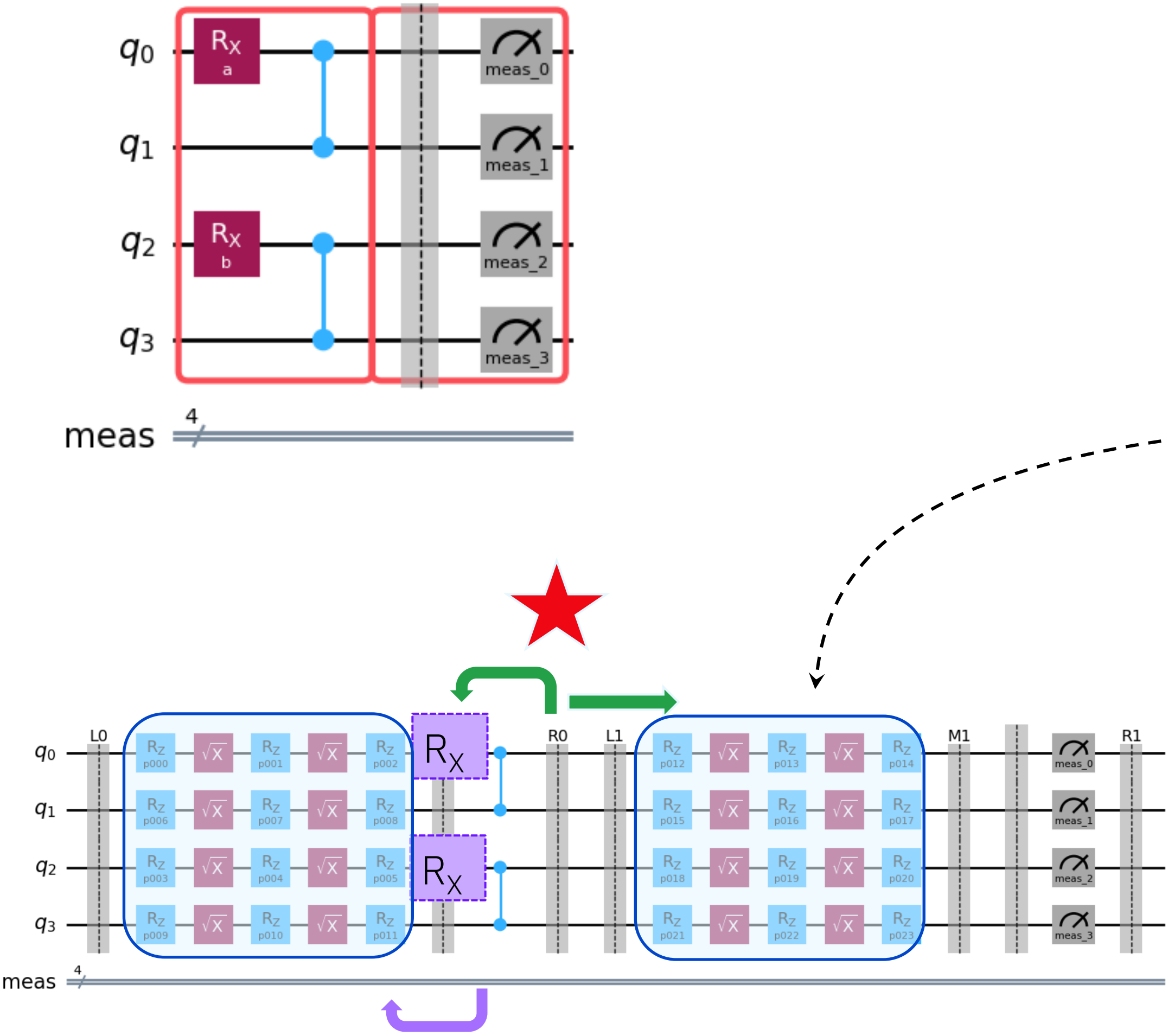
Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

Template circuits: WYSIWYG

Graph-based procedural representation of a distribution over template parameters & post-processing requisites.



3.1 Inspect:

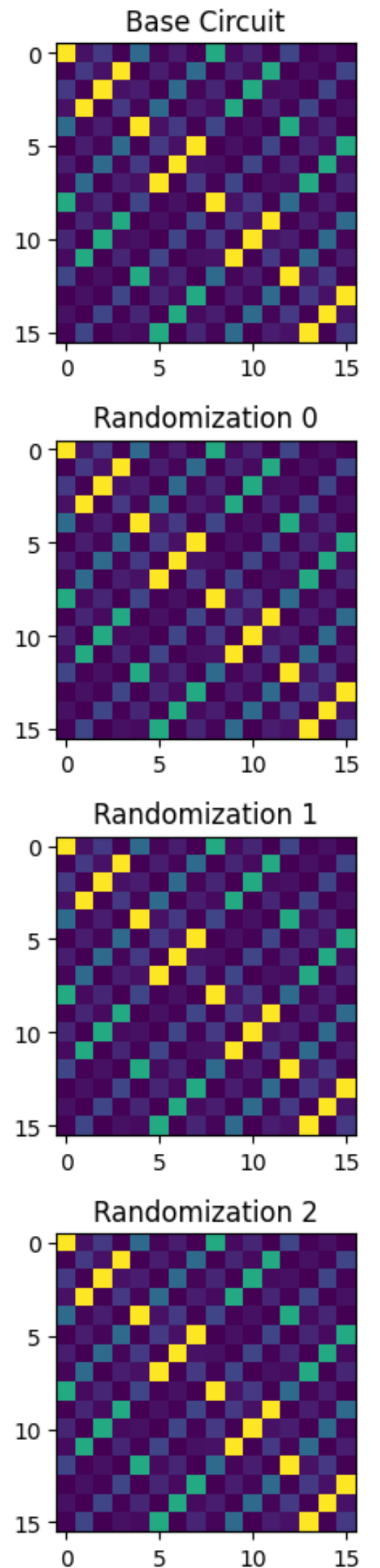
Samplex I/O & Sampling

```
Operator(circuit)
```

```
Operator(template.assign_parameters(output["parameter_values"][0]))
```

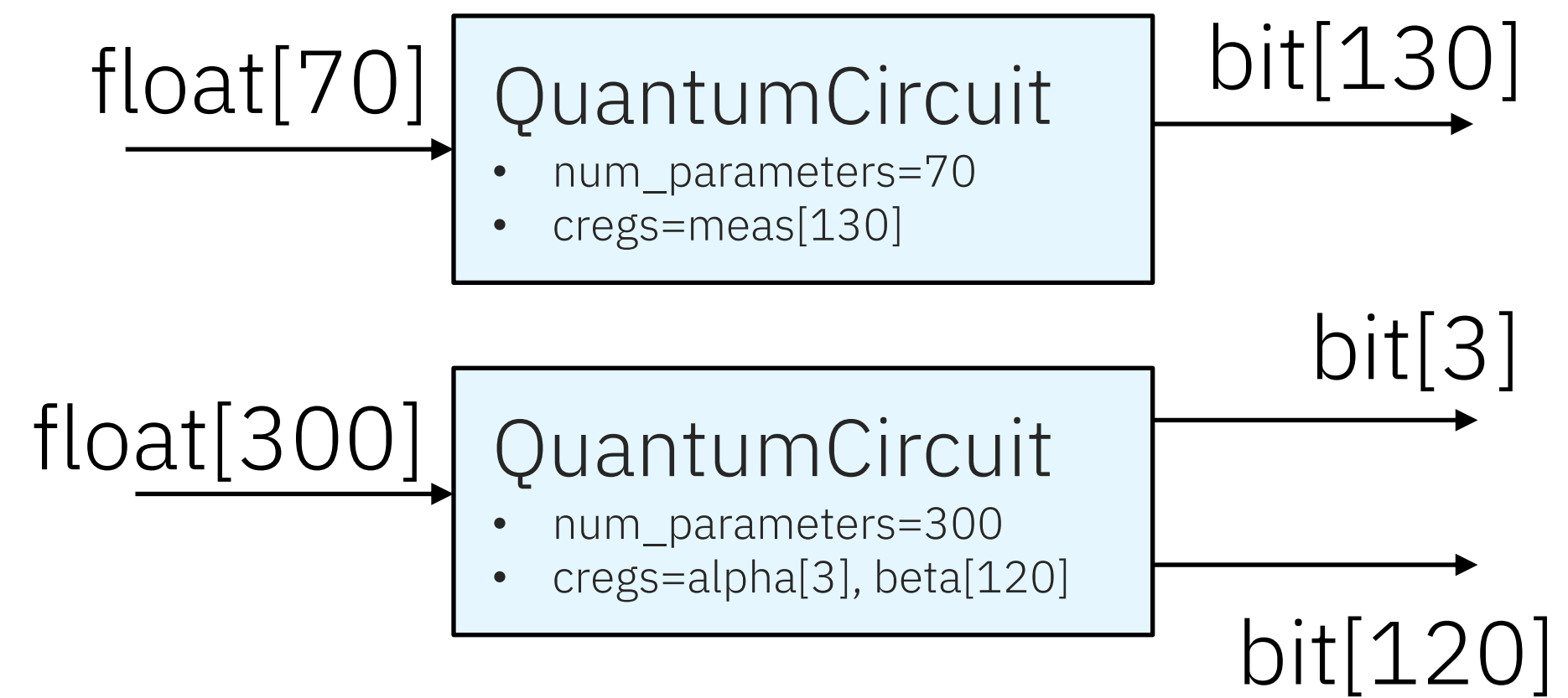
```
Operator(template.assign_parameters(output["parameter_values"][1]))
```

```
Operator(template.assign_parameters(output["parameter_values"][2]))
```



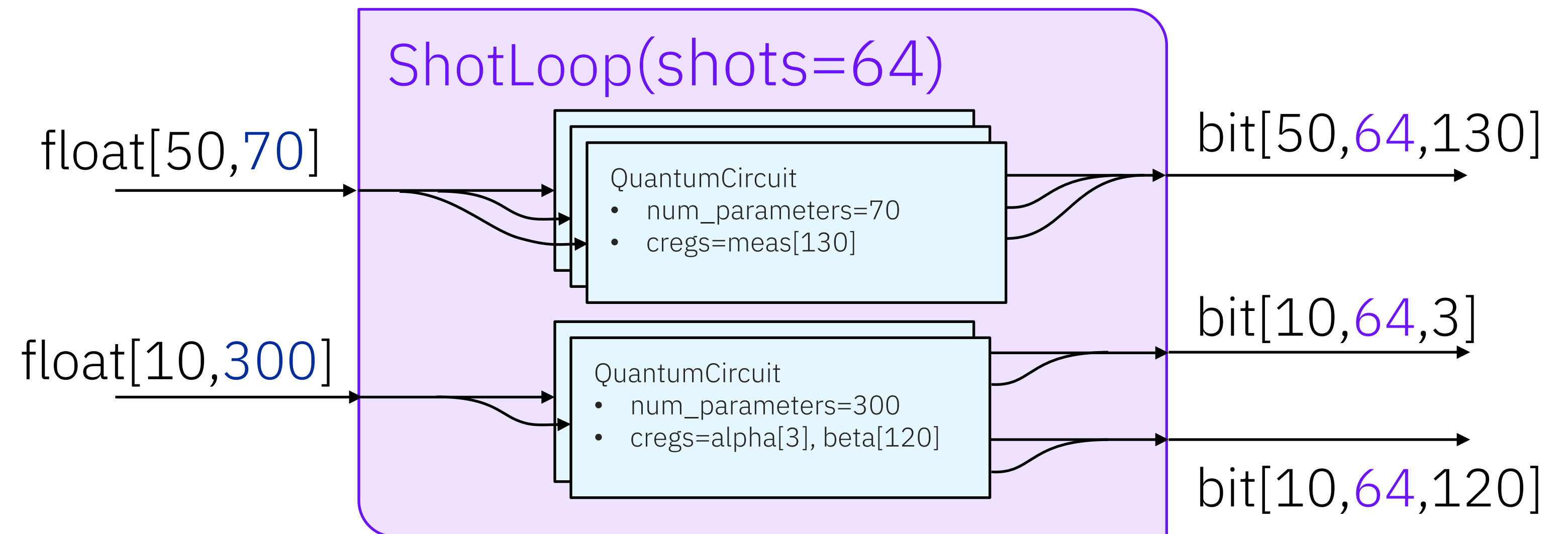
4.Execute

Randomize in the runtime



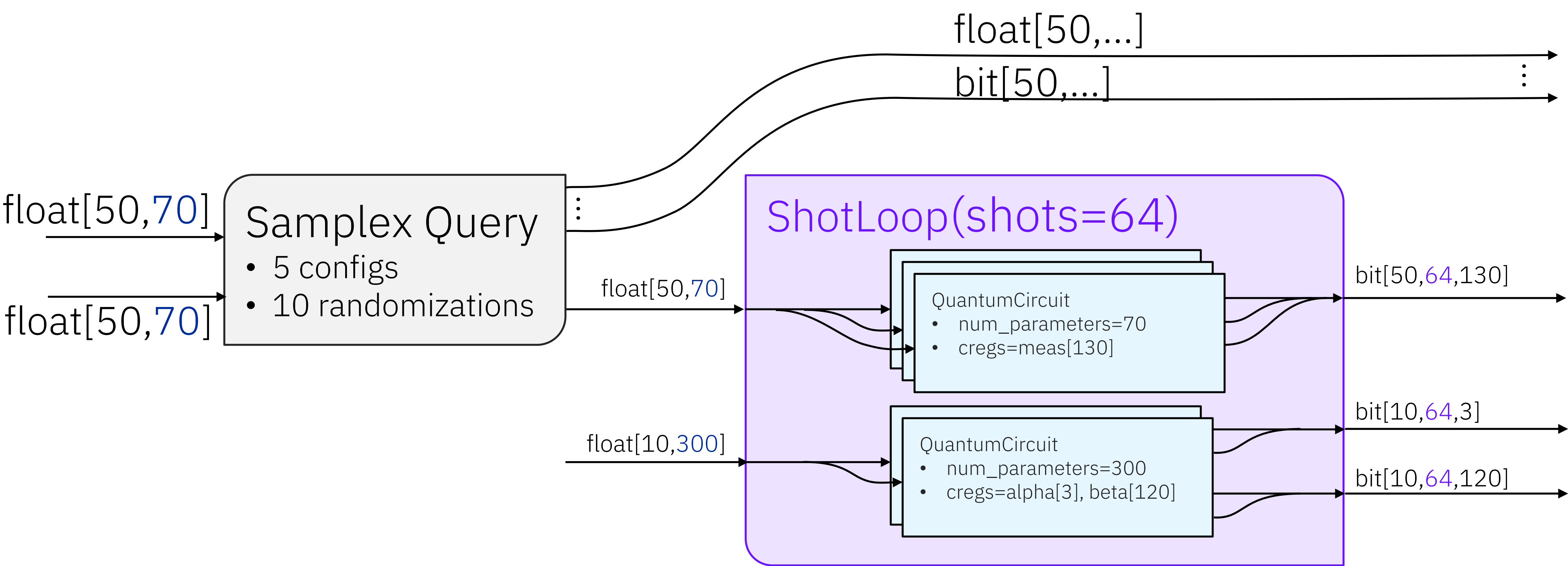
4. Execute

Randomize in the runtime



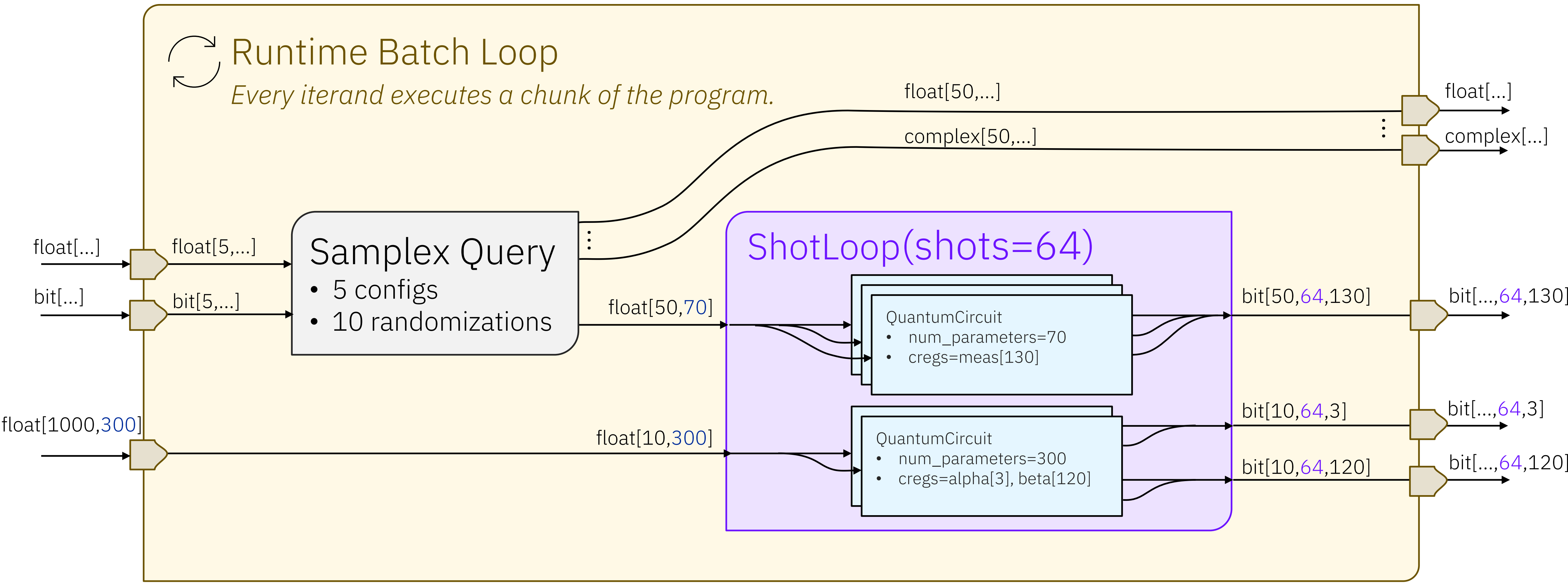
4. Execute

Randomize in the runtime



4. Execute

Randomize in the runtime



4.Execute

Randomize in the runtime

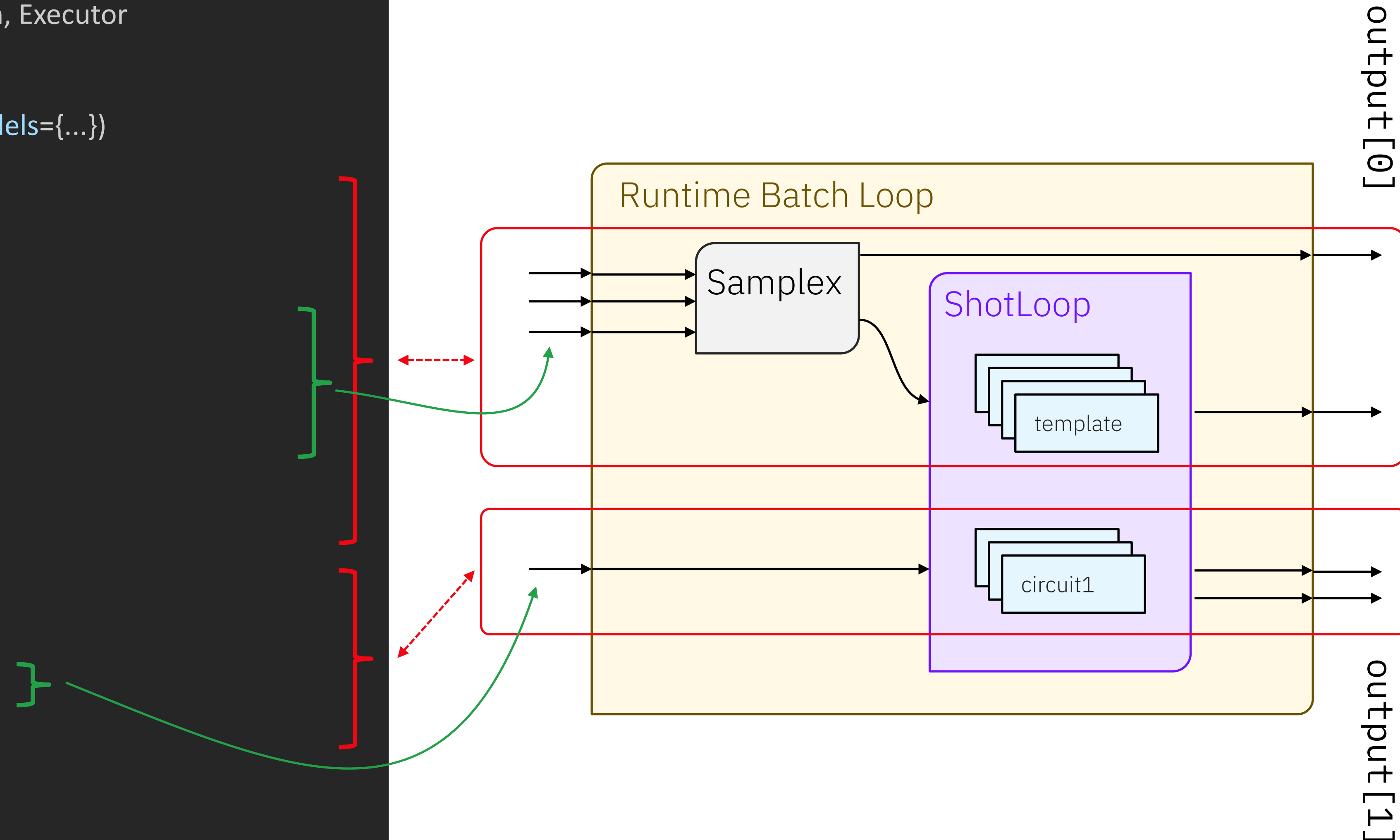
```
from qiskit_ibm_runtime import QuantumProgram, Executor
```

```
program = QuantumProgram(shots=64, noise_models={...})
```

```
program.append(  
    template,  
    samplex=samplex,  
    samplex_arguments={  
        "parameter_values": <param array>,  
        "basis_changes.meas0": <basis change array>,  
        "local_scales.mod_ref0": <scale array>  
    },  
    shape=(200,1,1,1)  
)
```

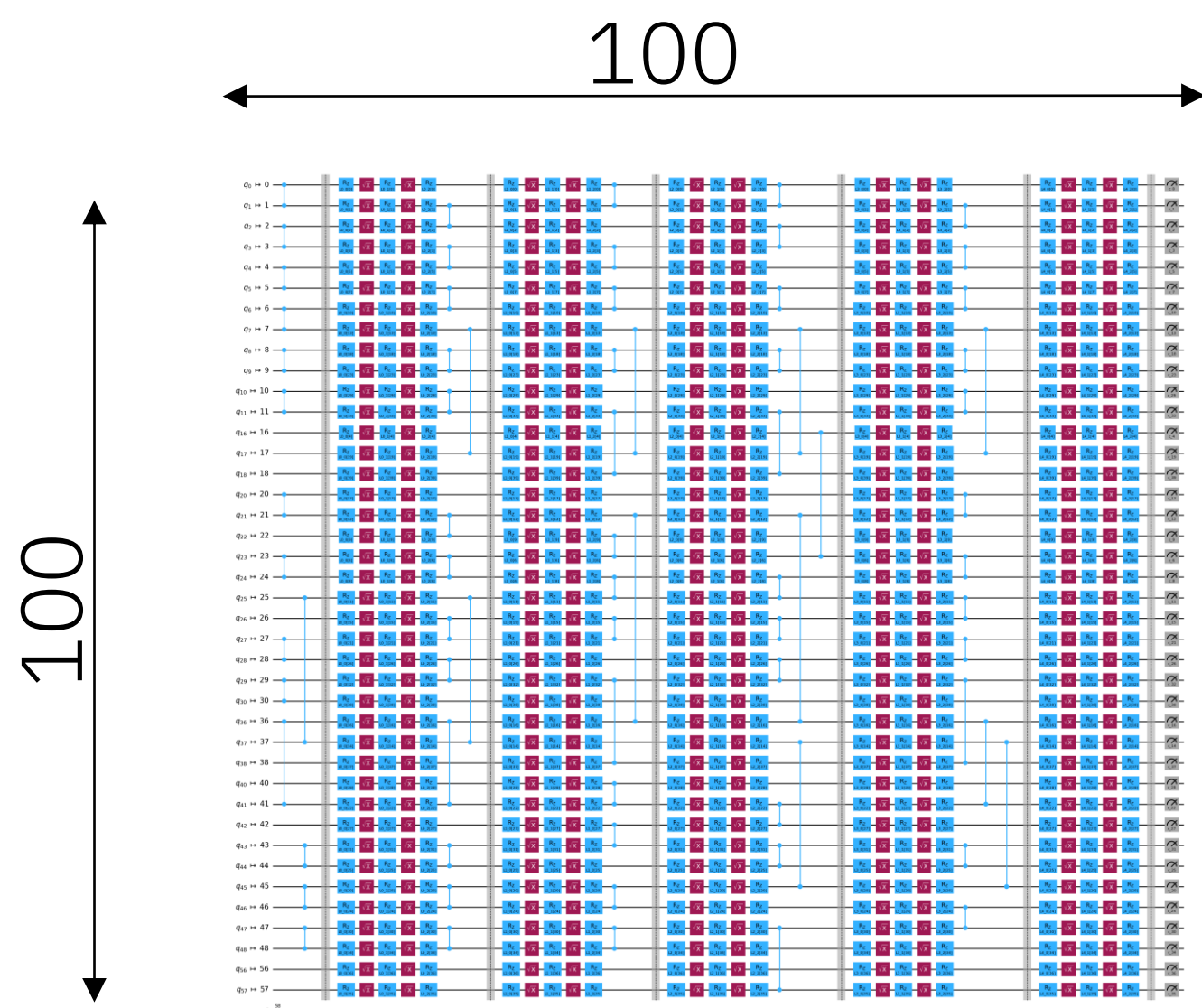
```
program.append(  
    circuit1,  
    circuit_arguments=<param array>  
)
```

```
executor = Executor(backend_or_session)  
job = executor.run(program)
```



4. Execute

With performance



- num_randomizations=5000
- Shots_per_randomization=100

Sampler

```
pub = (circuit, np.zeros(circuit.num_parameters))

with Session(backend) as session:
    sampler = Sampler(session)
    sampler.options.twirling.enable_gates = True
    sampler.options.twirling.enable_measure = True
    sampler.options.twirling.num_randomizations = 5000
    sampler_job = sampler.run([pub], shots=50000 * 100)

spans = sampler_job.result()
spans = spans.metadata["execution"]["execution_spans"]
spans = spans.sort()

sum_size = sum(span.size for span in spans)
size = sum_size - spans[0].size

end_of_last = spans.stop
end_of_first = spans[0].stop

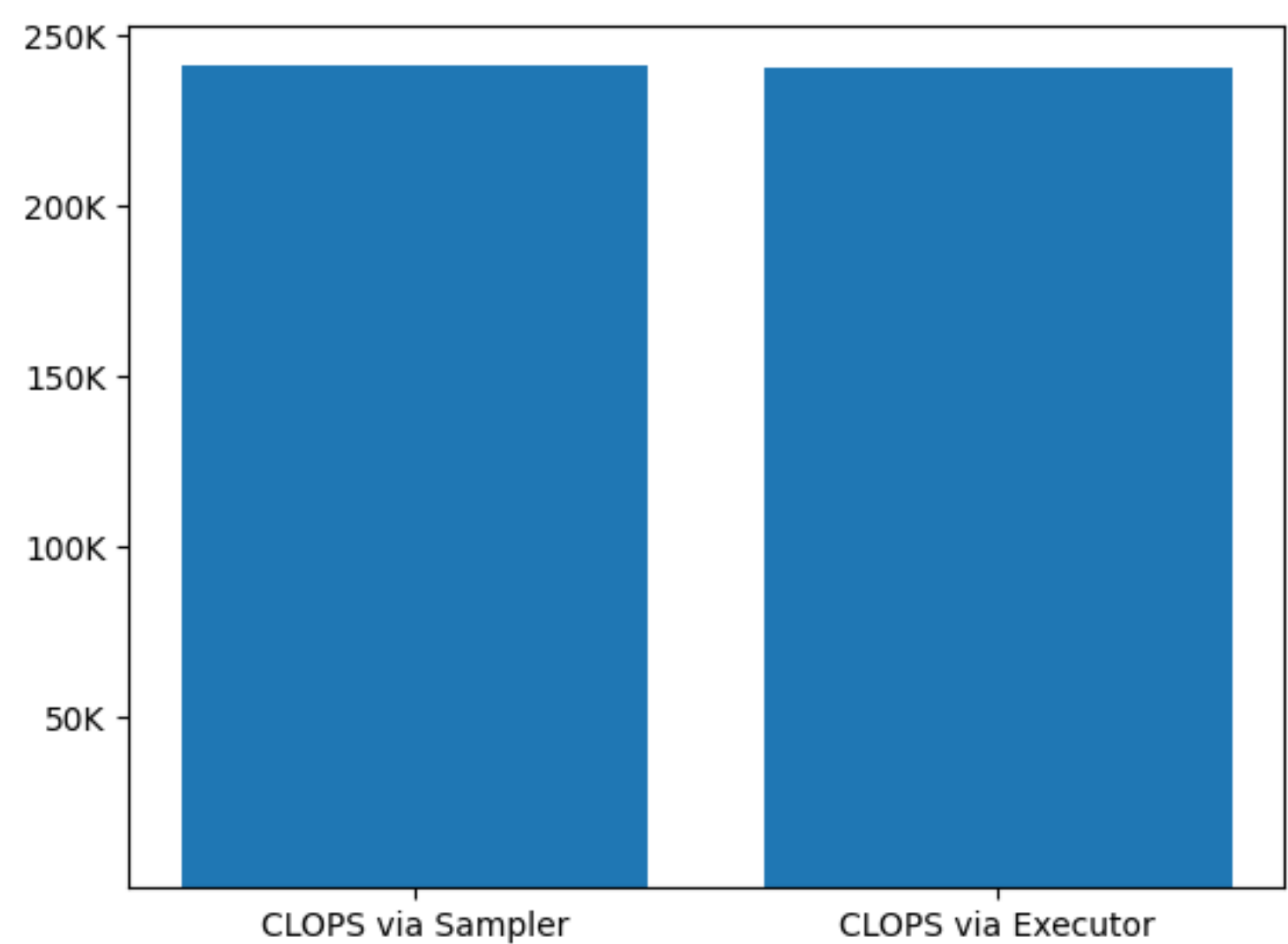
clops = (
    (size * num_layers) /
    (end_of_last - end_of_first).total_seconds()
)
```

Executor

```
with Session(backend) as session:
    executor = Executor(session)
    executor_job = executor.run(program)

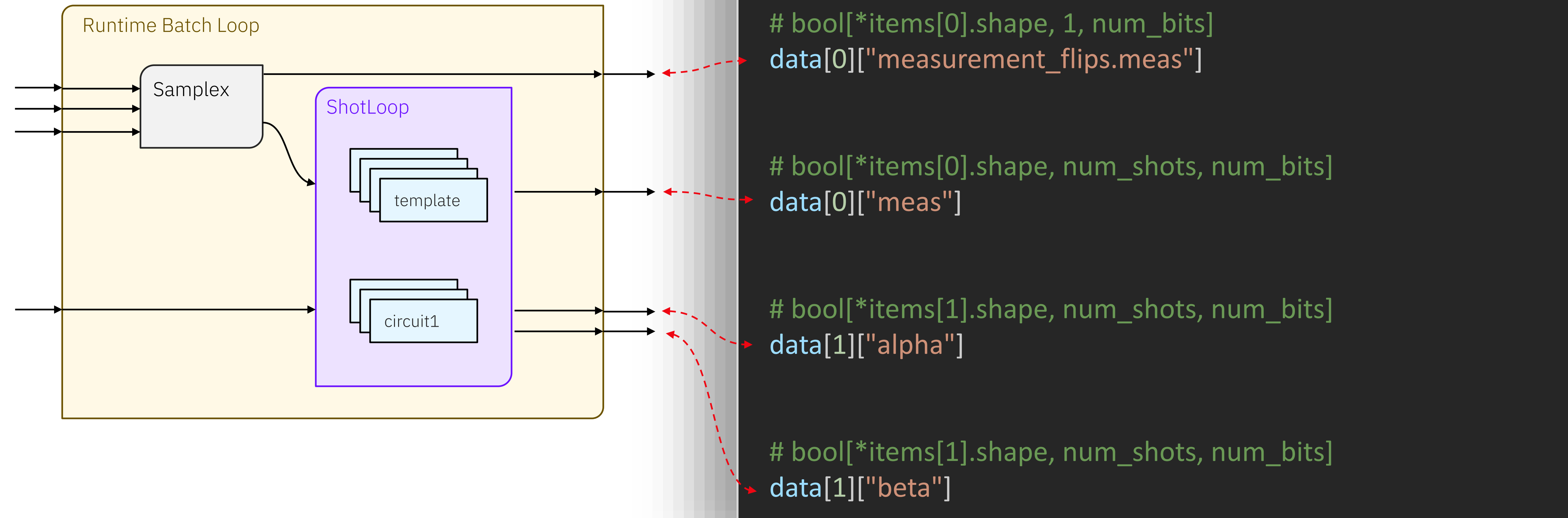
spans = executor_job.result().metadata.chunk_timing
first_span = min(spans, key=lambda span: span.start)

for span in spans:
    # ... (code for calculating CLOPS via Executor) ...
    (shots) /
    total_seconds()
```



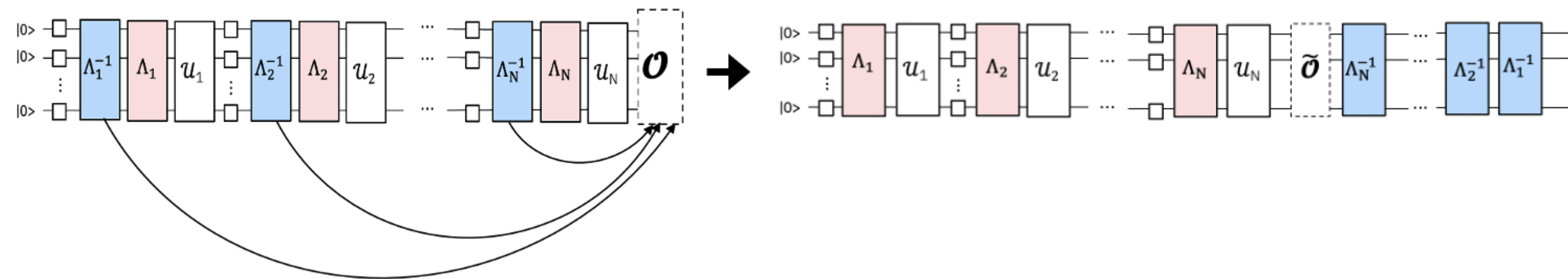
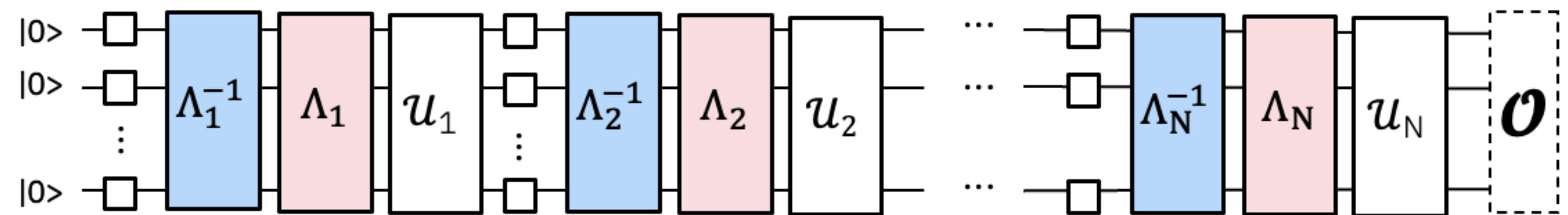
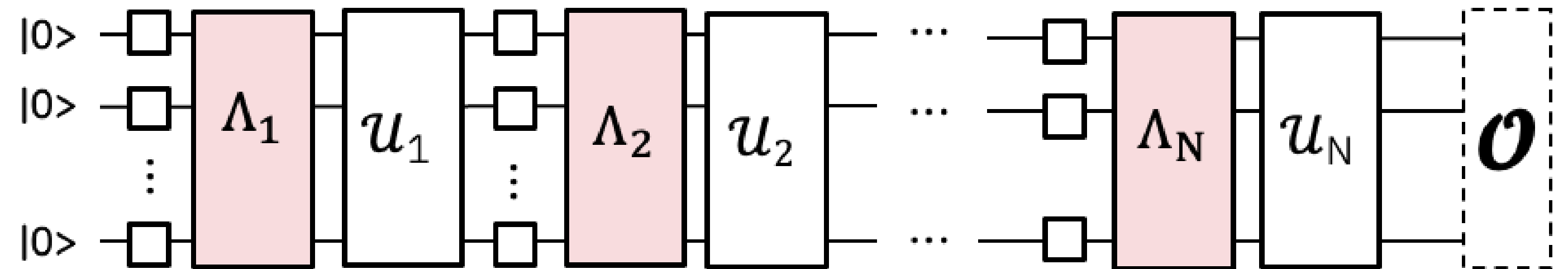
5. Post-process

Arrays of Data



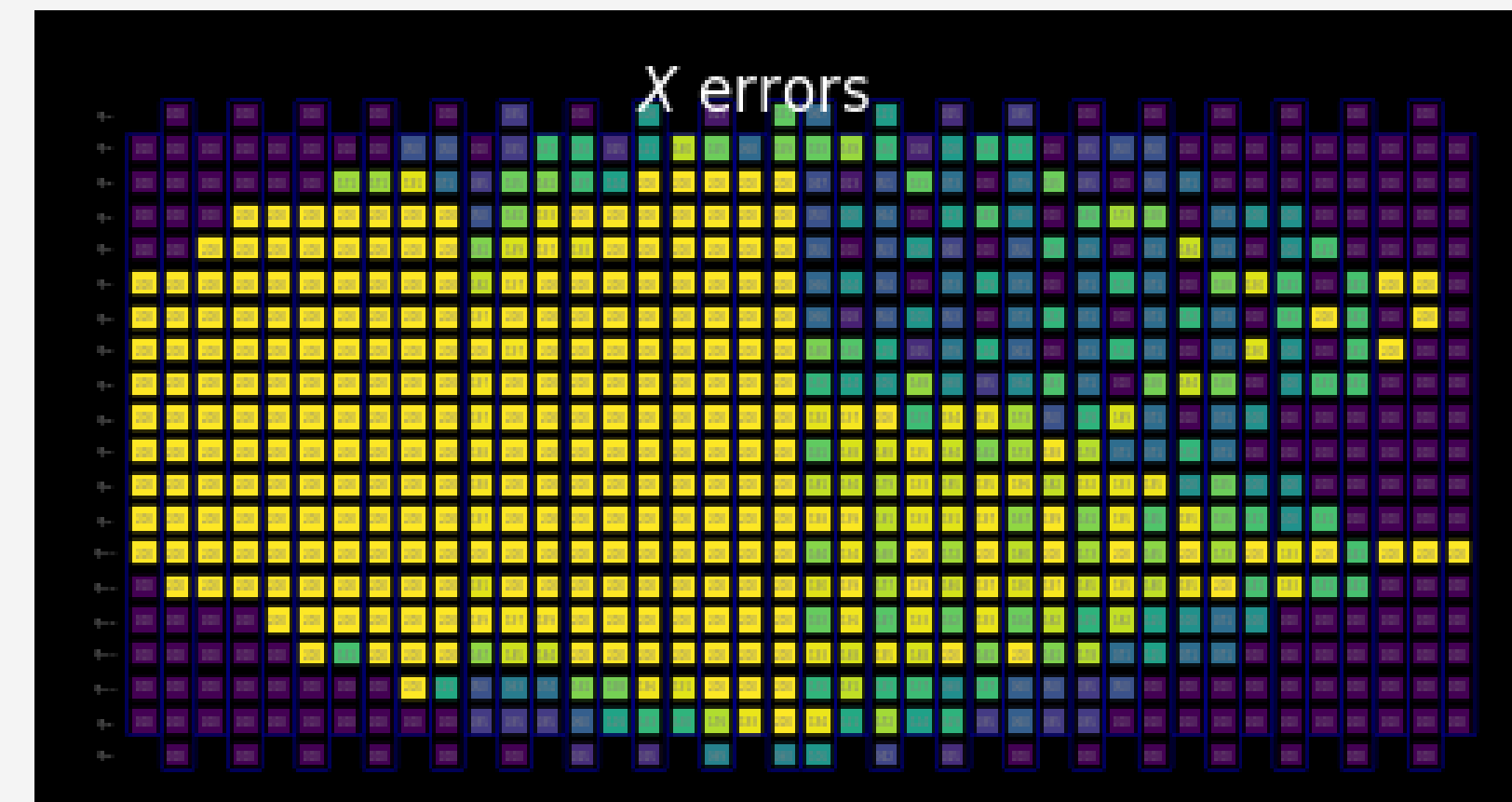
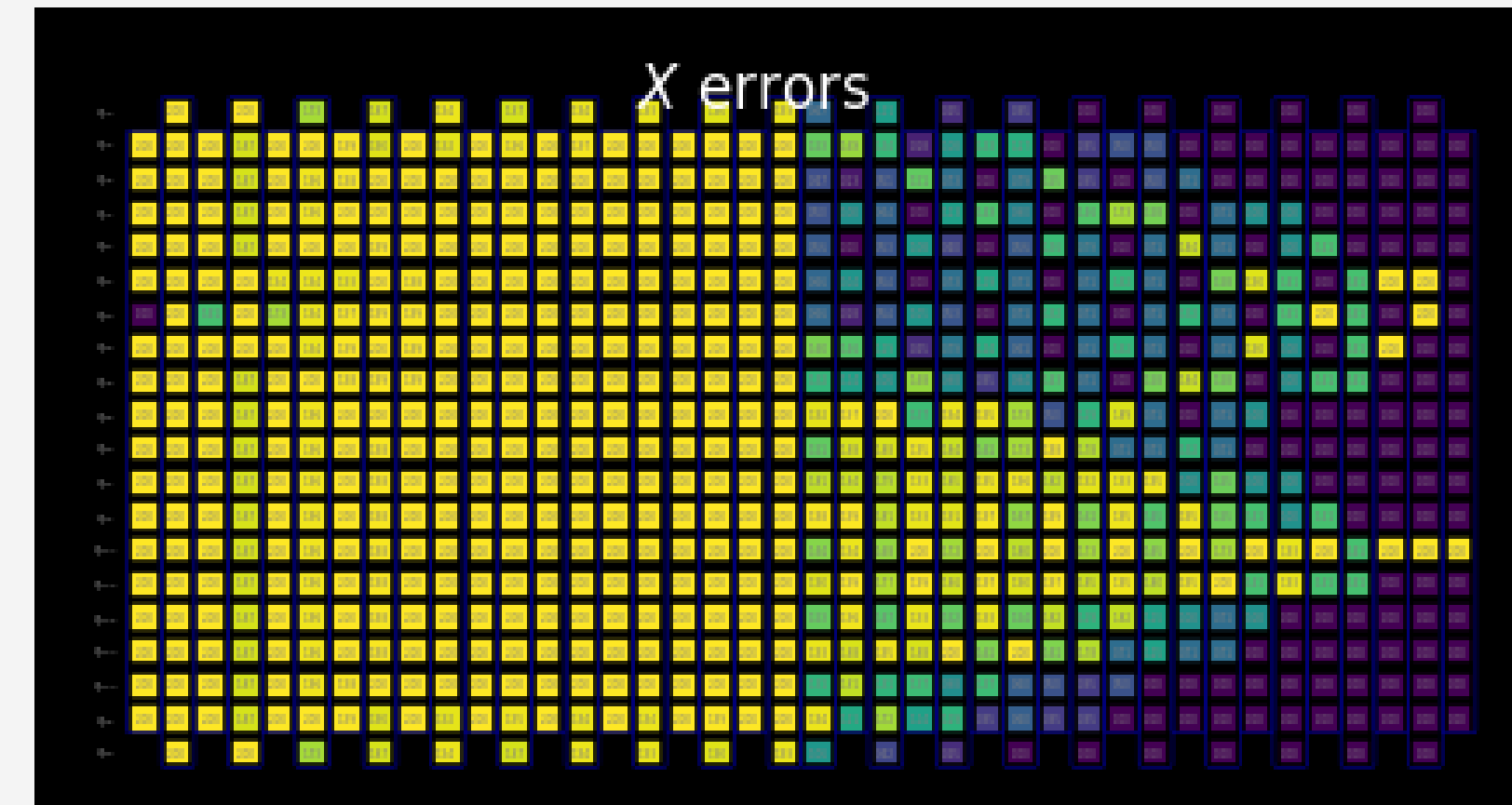
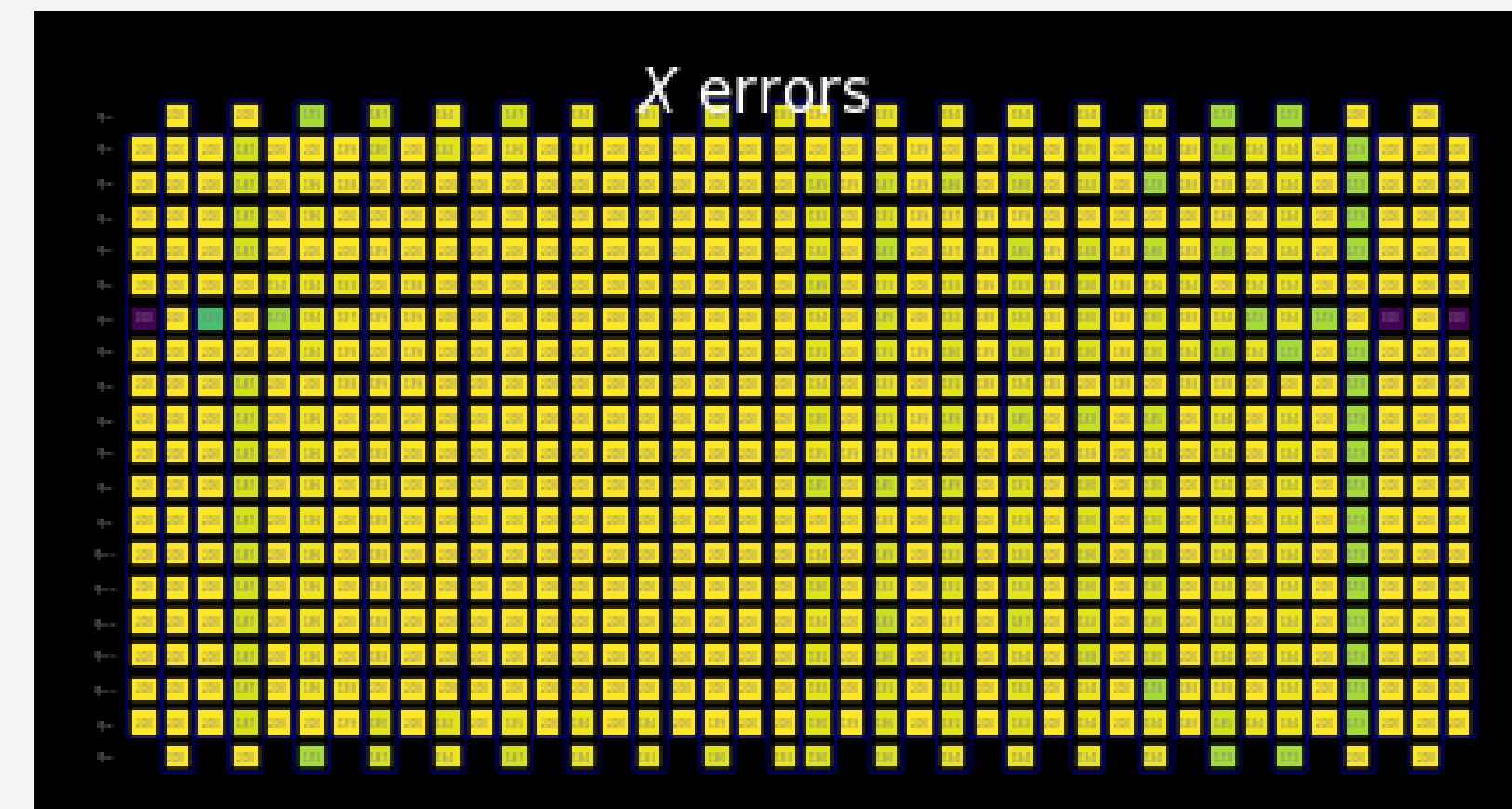
Propagated noise absorption (PNA) Addon

- Similar to PEC we need to learn the noise channels first.
- As a second step we calculate the inverse of the noise.
- Now instead of implementing the inverse noise in the circuit, we propagate the observables through the inverse noise channels.
- Measuring the new observable against the noisy state mitigates the learned noise.



Shaded Lightcones Addon (SLC)

- Shaded lightcones is an evolution of probabilistic error cancellation making use of samplomatic and the executor primitive.
- Main idea: Calculate tight bounds on how much error channels can bias the final results in order to target the application of error mitigation.
- First errors are forward propagated (bound effect on observables)
- Then backwards propagated (bound effect on initial state).
- The merged bounds are updated with learned noise rates.
- Mitigate noise based on the estimated impact on the bias.



Short repetition: Error Mitigation, Suppression, Correction & Detection

Error Suppression: Working closer to the hardware and with physics to prevent errors before they happen (e.g. working on pulse sequences, etc.)

Error Mitigation: Using strategic classical techniques (software/algorithmic approaches) to work around errors without targeting the underlying hardware, often in post-processing

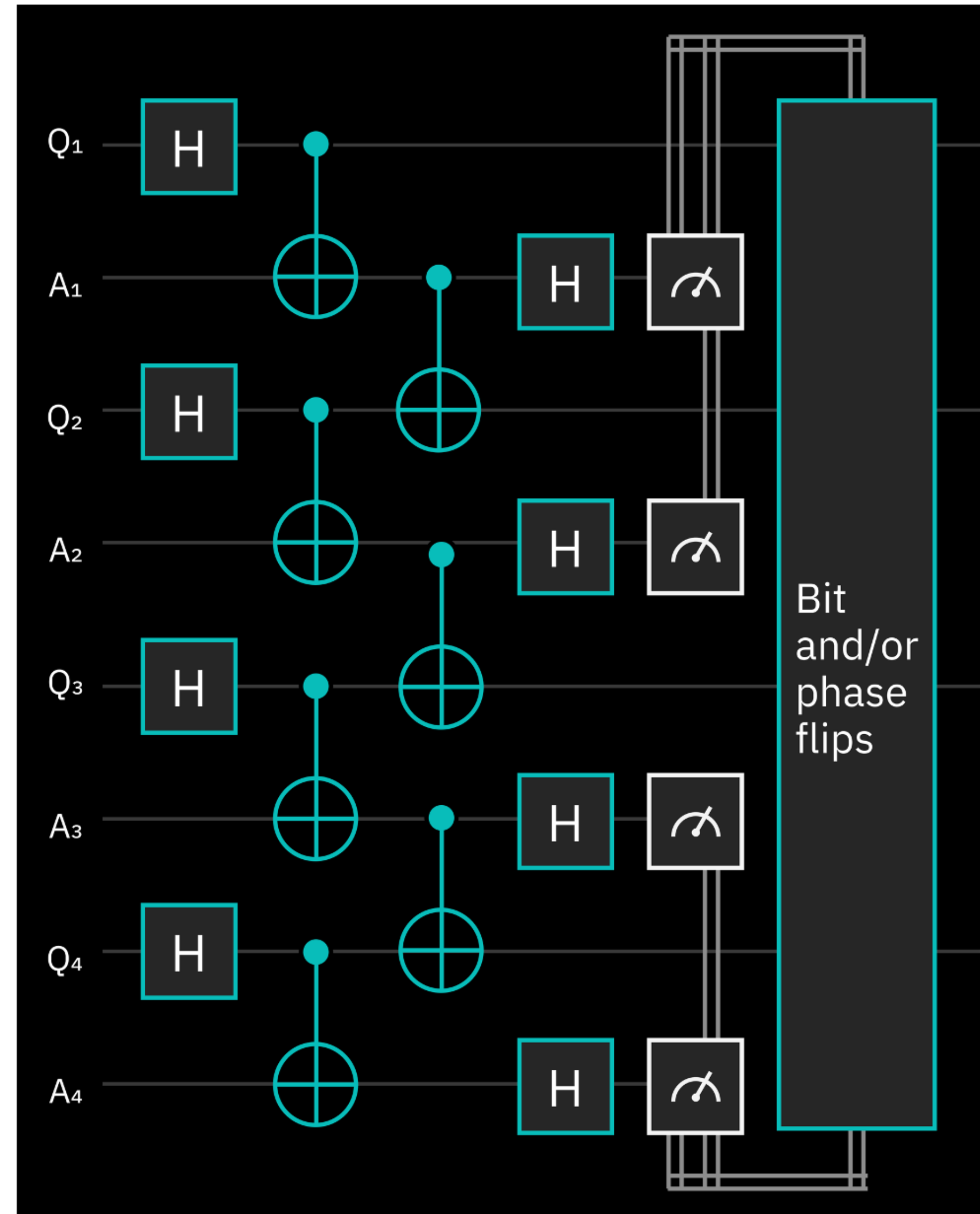
Error Detection: Using extra qubits to detect whether an error has occurred but not correcting it

Error Correction: Using quantum codes with extra qubits to actively detect and correct errors

From error detection to error correction

1. We need to measure errors during runtime as they happen.
2. We need to correct the errors according to the measurements.

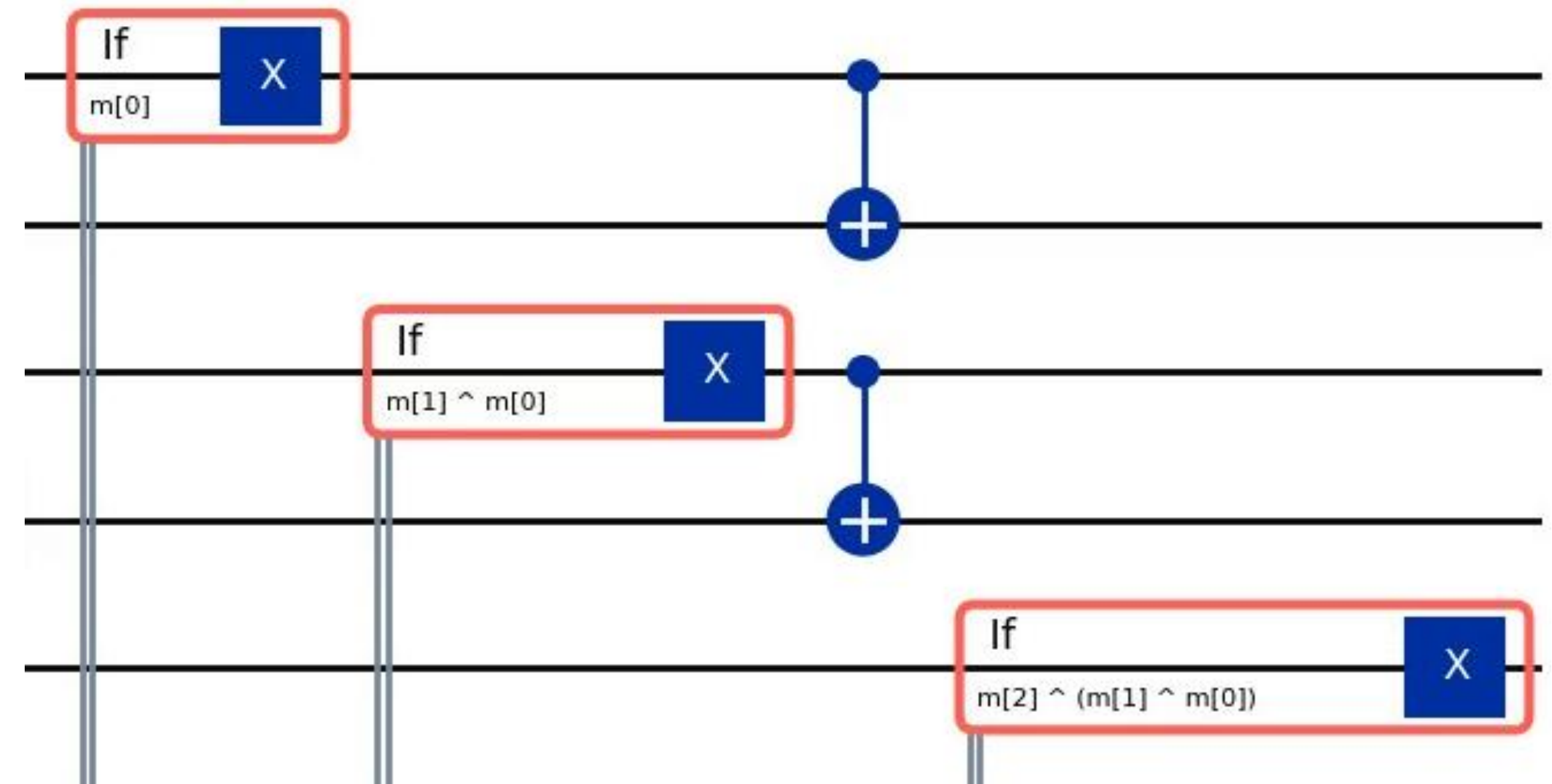
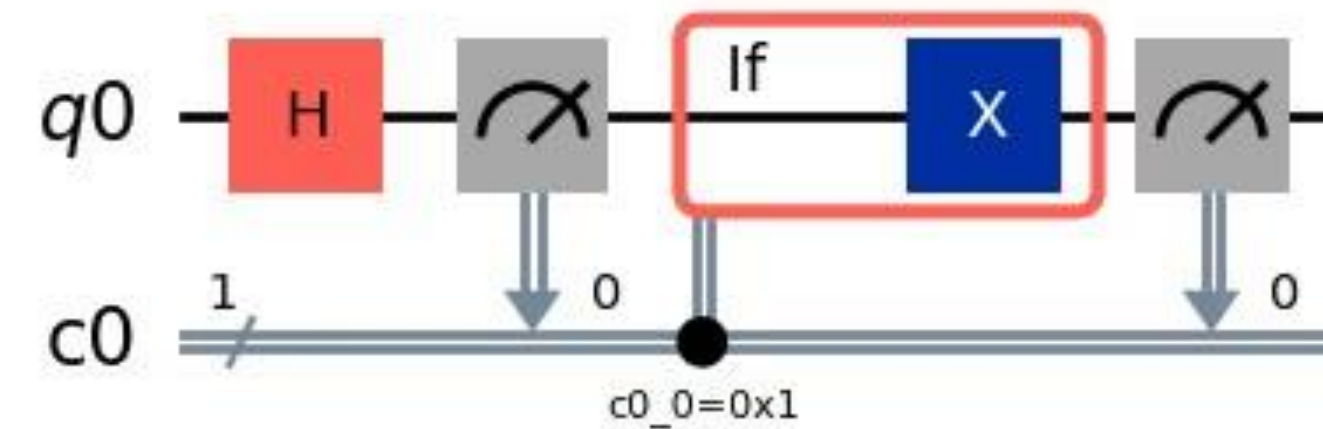
-> Dynamic Circuits



Mid-circuit Measurements

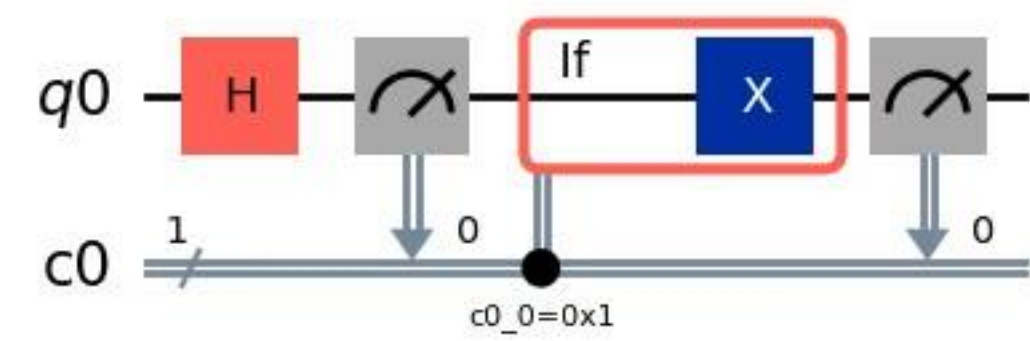
What are Mid-circuit Measurements ?

- They work like a normal measurement
- Done during calculation
- Result can be used for branching
- The same qubit can be used for measurement several times!
- We can use classical logic!

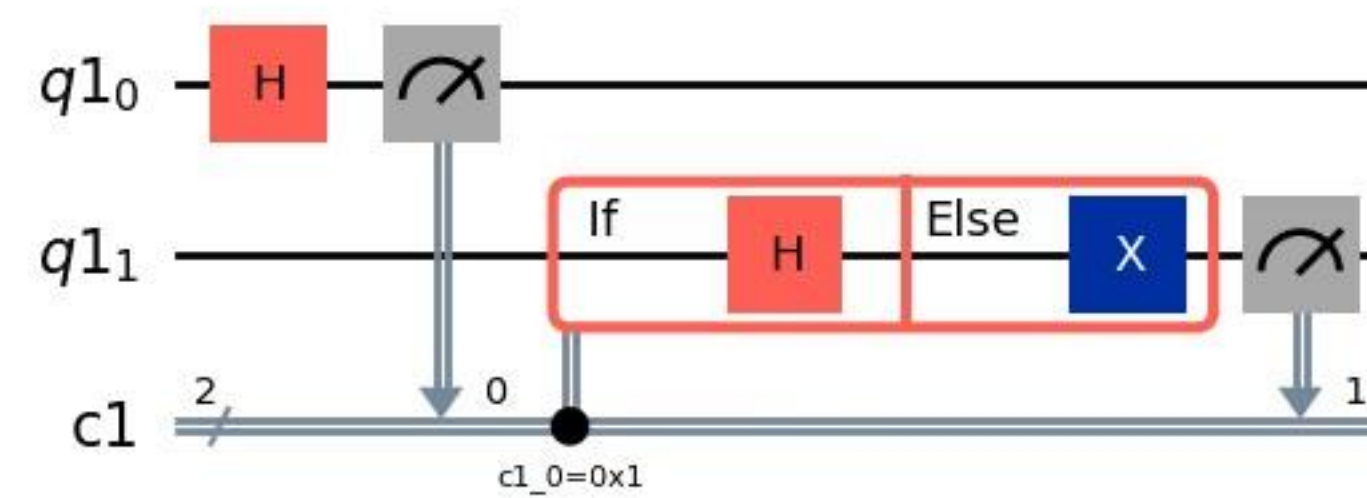


What can we do with mid circuit measurements

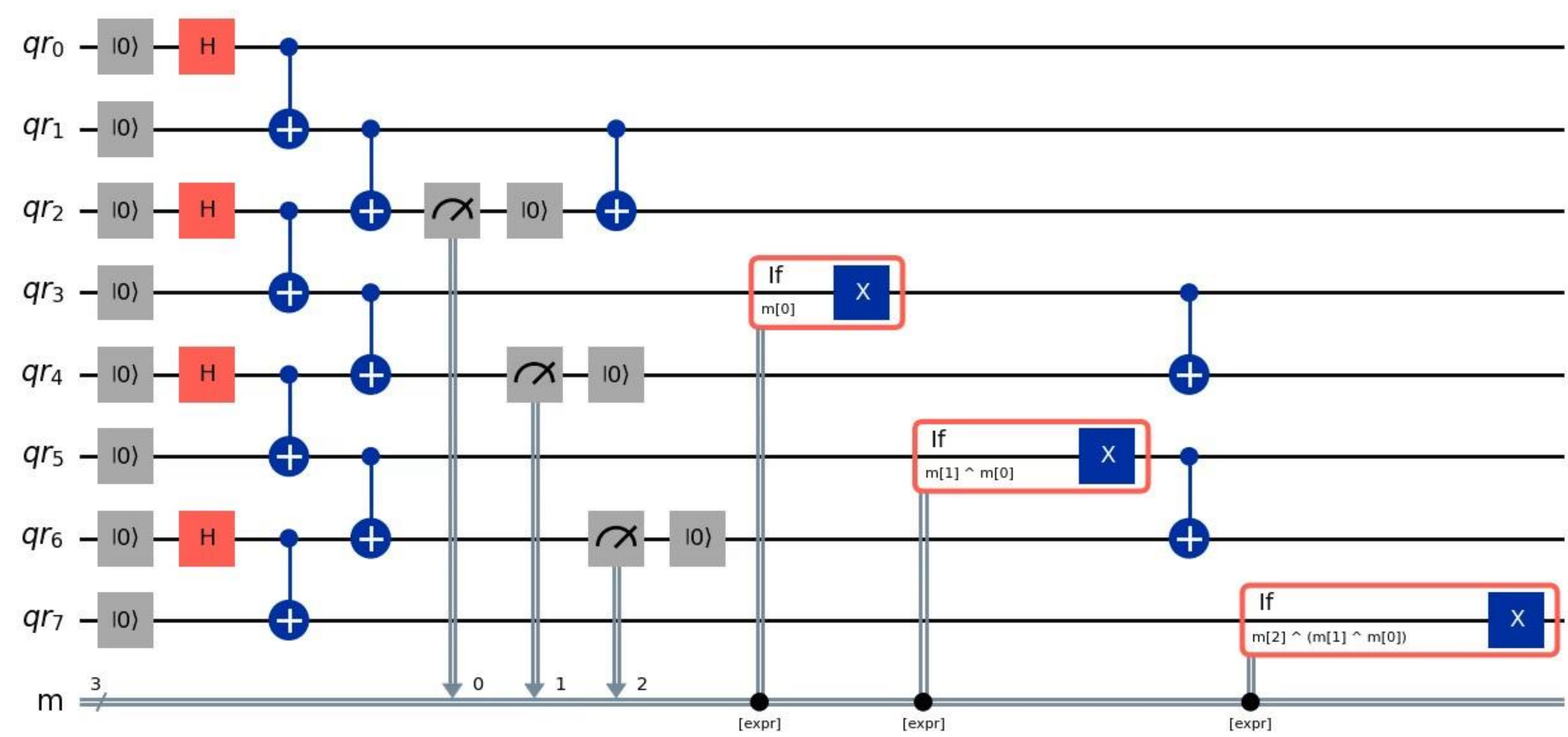
Reset



Create Entanglement



Calculate parity



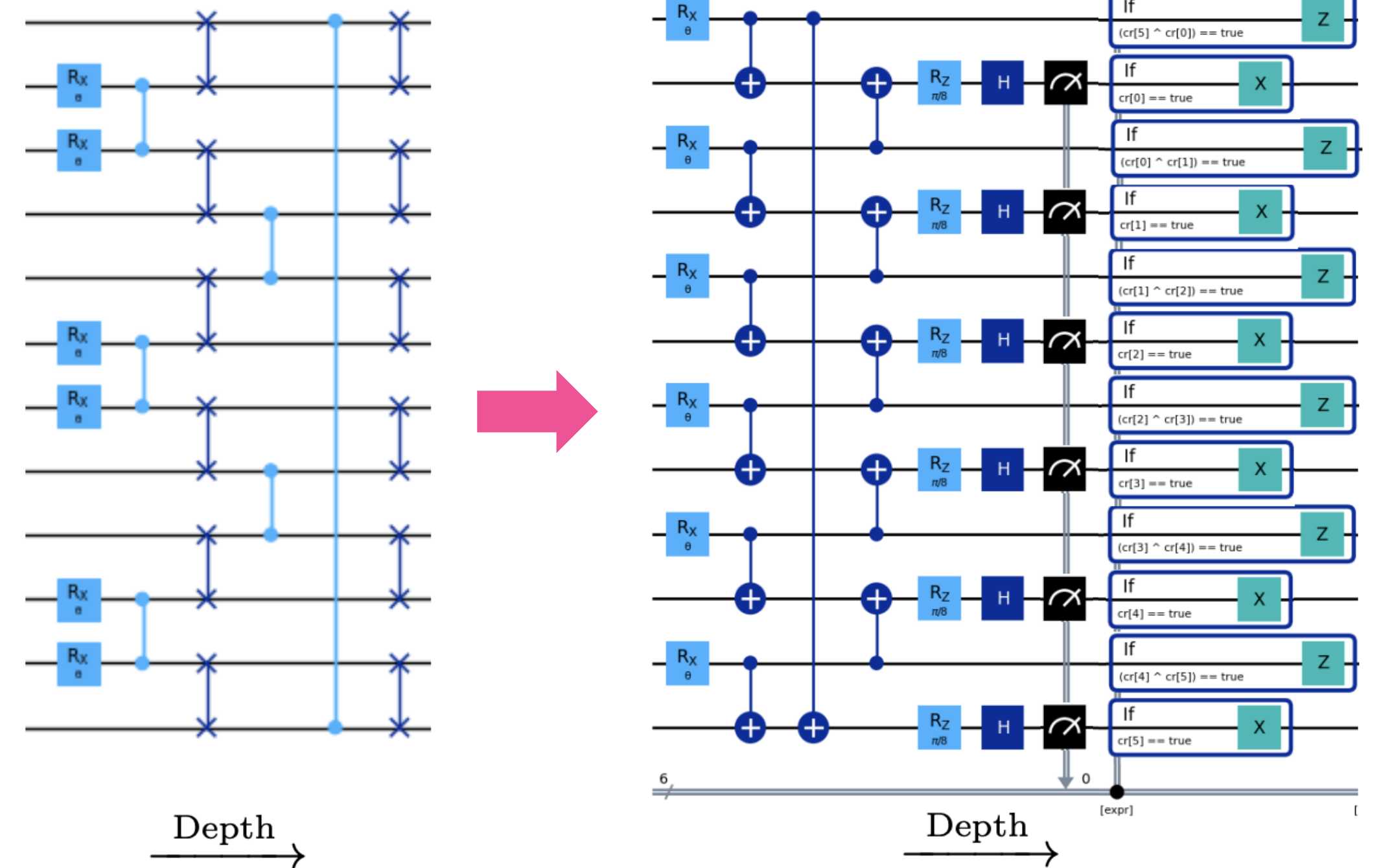
Power of utility-scale dynamic circuits

58% of reduction in two-qubit gates

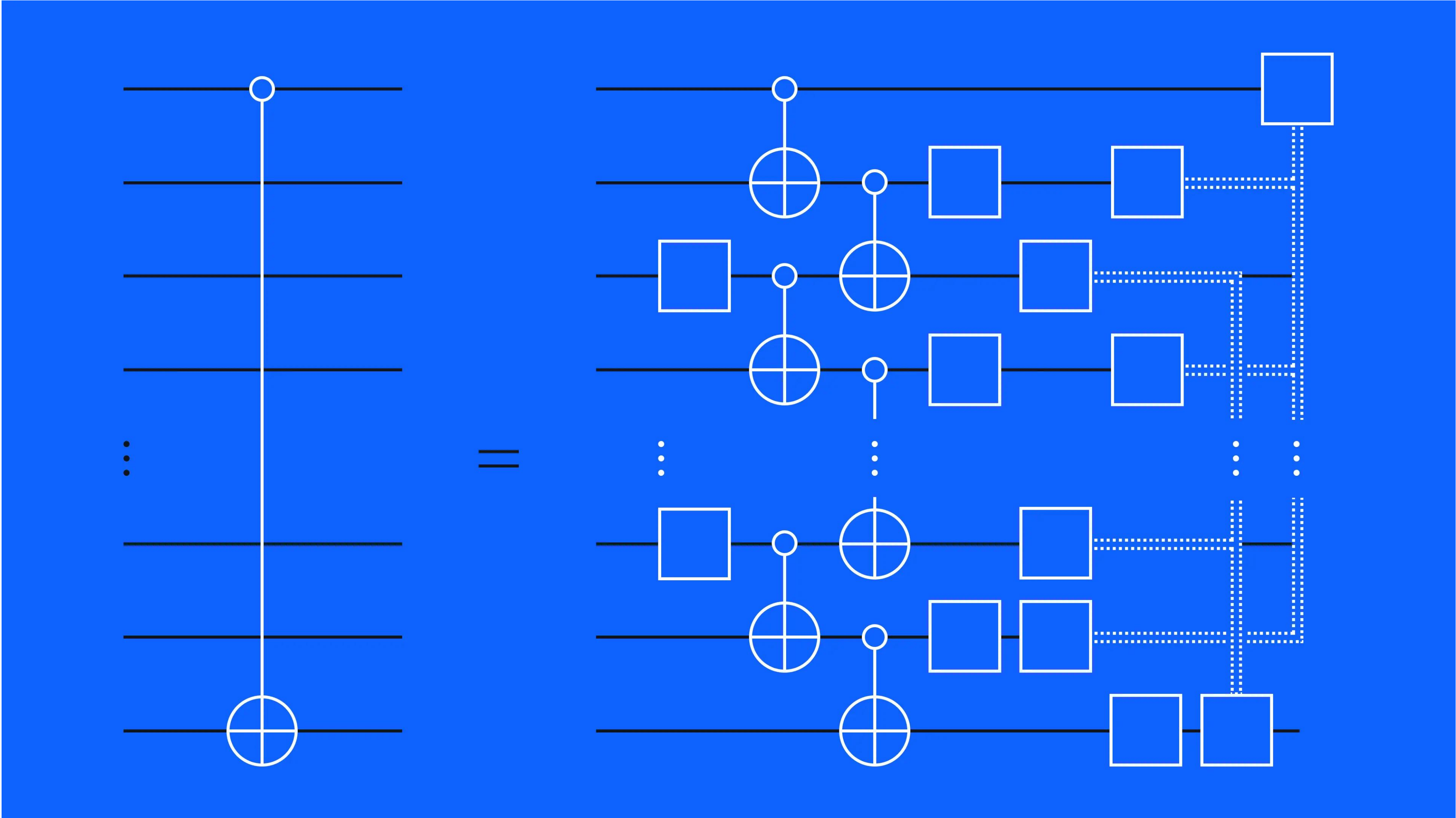
and increasing the accuracy by 24%
for a probe observable

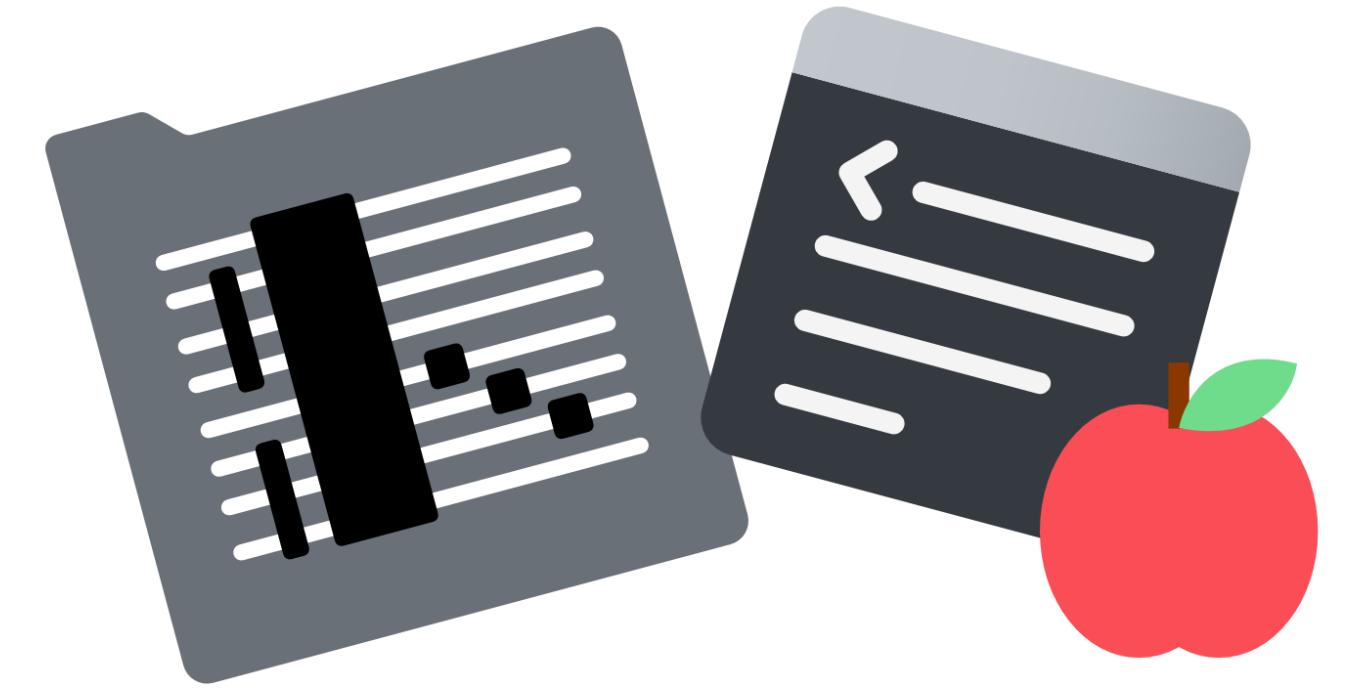
Key improvements

- 568ns classical feedback time
- Multiple measurement support
- Stretches and parallelized branch execution at scale
- Full feature support via Qiskit SDK



Long Range Entanglement





Further reading

Executor Primitive and new Addons

<https://quantum.cloud.ibm.com/docs/en/guides/directed-execution-model>

<https://github.com/Qiskit/samplomatic/>

<https://github.com/Qiskit/qiskit-addon-pna>

<https://github.com/Qiskit/qiskit-addon-slc>

Lightcone shading for classically accelerated quantum error mitigation

<https://arxiv.org/html/2409.04401v1>

Dynamic Circuits:

Dynamic Circuit Implementation

<https://quantum.cloud.ibm.com/docs/en/guides/classical-feedforward-and-control-flow>

Long Range Entanglement

<https://www.ibm.com/quantum/blog/long-range-entanglement>

<https://arxiv.org/abs/2308.13065>

Hands On!